PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
ENGENHARIA DE SOFTWARE

EDUARDO HASHIMOTO YONEGURA
GABRIEL ANDRÉ MORO
GUILHERME SAKAGAMI SPITZNER
JOÃO PEDRO PIRES ANTUNES
LETÍCIA BEATRIZ SILVA DE MORAIS

HELP ME: DISPOSITIVO DE ALERTA PARA VÍTIMAS DE VIOLÊNCIA DOMÉSTICA COM ESP32

CURITIBA
2026

EDUARDO HASHIMOTO YONEGURA
GABRIEL ANDRÉ MORO
GUILHERME SAKAGAMI SPITZNER
JOÃO PEDRO PIRES ANTUNES
LETÍCIA BEATRIZ SILVA DE MORAIS

HELP ME: DISPOSITIVO DE ALERTA PARA VÍTIMAS DE VIOLÊNCIA DOMÉSTICA COM ESP32

Relatório técnico apresentado à disciplina de Conectividade em Sistemas Ciberfísicos do curso de Engenharia de Software da Escola Politécnica da Pontifícia Universidade Católica do Paraná, como requisito parcial de avaliação do Trabalho Discente Efetivo (TDE).

Orientador: Prof. Fábio Bettio.

CURITIBA
2026

RESUMO

O HELP ME é um sistema para que uma vítima de violência doméstica peça ajuda com discrição. O protótipo usa um ESP32 ligado ao Wi-Fi, que funciona como servidor HTTP na porta 80. O botão e o microfone digital INMP441 são os sensores, e um motor de vibração é o atuador. Quando a usuária segura o botão por um segundo, o dispositivo vibra uma vez e abre um pré-alerta de 20 segundos. Nesse tempo ela pode cancelar pelo Telegram, com uma senha que define ao adquirir o dispositivo. Sem cancelamento, a ocorrência chega à central de segurança, uma página web servida pelo próprio ESP32, que recebe o áudio gravado como prova, a conversa com a usuária e a localização em tempo real enviada do celular dela pelo Telegram. Uma senha de coação finge o cancelamento e mantém o alerta ativo em modo silencioso. O sistema grava um log na memória flash, com níveis INFO, AVISO e ERRO e data e hora obtidas por NTP, e reconecta ao Wi-Fi quando a rede cai. Os testes de ping, de acesso à porta 80, de rotas com navegador e curl, de estresse com 100 requisições, de queda de rede e de reinício confirmaram o funcionamento do servidor e a persistência dos dados. O protótipo mostra que componentes de baixo custo bastam para um sistema ciberfísico de alerta que usa a pilha TCP/IP e a arquitetura cliente-servidor.

Palavras-chave: ESP32; servidor HTTP; sistemas ciberfísicos; Telegram; violência doméstica.

LISTA DE FIGURAS

Figura 1 – Arquitetura do HELP ME....13
Figura 2 – Mapa de ligações na protoboard....15
Figura 3 – Protótipo físico montado na protoboard, antes da case 3D....16
Figura 4 – Estados e regras de funcionamento....17
Figura 5 – Central de segurança: visão geral e painel do dispositivo físico....20
Figura 6 – Cadastro de usuárias na central....21
Figura 7 – Central com uma ocorrência em andamento....21
Figura 8 – Localização enviada pela usuária: mapa na central e confirmação no Telegram....22
Figura 9 – Telegram da usuária: comandos /status e /teste, pré-alerta e cancelamento com senha....23
Figura 10 – Pastas com os códigos de teste....25
Figura 11 – Simulação no Wokwi....28
Figura 12 – ESP32 iniciando o servidor na porta 80....30
Figura 13 – Teste de ping....31
Figura 14 – Teste de acesso à porta 80....31
Figura 15 – Rota raiz e rota /sensor no navegador....32
Figura 16 – Log com os alertas disparados....32
Figura 17 – Queda e reconexão do Wi-Fi....33
Figura 18 – Log mantido depois de reiniciar....33
Figura 19 – Teste de estresse com 100 requisições....34
Figura 20 – Acesso pelo celular....35
Figura 21 – Inicialização, ping e acesso à porta 80 na versão final....36
Figura 22 – Comunicação cliente-servidor com curl....37
Figura 23 – Motor acionado pela rede e registro no log....37
Figura 24 – Pré-alerta na central com contagem regressiva para o cancelamento....38
Figura 25 – Atendimento: conversa vista na central e no Telegram da usuária....38

Figura 26 – Coação: com a senha de coação, o celular mostra "Alerta cancelado".....39

Figura 27 – Histórico e estatísticas com dados reais.....40

Figura 28 – Vídeo geral de demonstração: quadros e QR code do link.....41

LISTA DE QUADROS

Quadro 1 – Camadas do modelo TCP/IP no HELP ME.....10
Quadro 2 – Projetos semelhantes.....12
Quadro 3 – Componentes e pinos.....14
Quadro 4 – Rotas do servidor HTTP do ESP32.....18
Quadro 5 – Equipe.....24
Quadro 6 – Bibliotecas do firmware.....26
Quadro 7 – Problemas encontrados e soluções.....27
Quadro 8 – Resultados da primeira rodada.....29
Quadro 9 – Resultados da segunda rodada.....35
Quadro 10 – Atendimento aos requisitos do TDE.....42

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface (interface de programação)
GPIO	General Purpose Input/Output (pino de uso geral)
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP sobre TLS (HTTP seguro)
I2S	Inter-IC Sound (barramento de áudio digital)
IP	Internet Protocol
JSON	JavaScript Object Notation
MAC	Media Access Control (endereço físico)
NTP	Network Time Protocol
OSI	Open Systems Interconnection
RSSI	Received Signal Strength Indicator (força do sinal)
TCP	Transmission Control Protocol
TDE	Trabalho Discente Efetivo
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAV	Waveform Audio File Format

SUMÁRIO

1 INTRODUÇÃO..... 9

2 FUNDAMENTAÇÃO TEÓRICA..... 10

2.1 Sistemas ciberfísicos..... 10

2.2 Modelo OSI e modelo TCP/IP..... 10

2.3 Arquitetura cliente-servidor..... 10

2.4 TCP e UDP..... 11

2.5 Sockets..... 11

2.6 Protocolo HTTP..... 11

2.7 Projetos semelhantes..... 11

3 ARQUITETURA DO SISTEMA..... 13

3.1 Visão geral..... 13

3.2 Componentes e ligações..... 13

3.3 Regras de funcionamento..... 16

3.4 Rotas do servidor (API)..... 18

3.5 Central de segurança (cliente web)..... 19

3.6 Canal da usuária (Telegram)..... 22

4 DESENVOLVIMENTO..... 24

4.1 Equipe..... 24

4.2 Etapas..... 24

4.3 Firmware..... 26

4.4 Problemas encontrados e soluções..... 27

4.5 Simulação no Wokwi..... 28

5 TESTES E VALIDAÇÃO..... 29

5.1 Ambiente de testes..... 29

5.2 Primeira rodada: rede e servidor (16/10/2026)..... 29

5.3 Segunda rodada: versão final..... 35

6 RESULTADOS.....42
7 CONCLUSÃO.....43
REFERÊNCIAS.....44
APÊNDICE A – PINAGEM DO ESP32.....46

1 INTRODUÇÃO

A violência contra a mulher no Brasil acontece, na maior parte das vezes, dentro de casa. Em 2025 o país registrou 1.571 feminicídios, o maior número da série histórica, e 62,5% das vítimas foram mortas na própria residência (FÓRUM BRASILEIRO DE SEGURANÇA PÚBLICA, 2026). A Lei Maria da Penha define a violência doméstica e prevê medidas de proteção (BRASIL, 2006), mas, no momento da agressão, pegar o celular, desbloquear e ligar para alguém pode ser impossível ou perigoso.

O HELP ME é um dispositivo pequeno, com um botão, que a vítima mantém por perto. Ao segurar o botão por um segundo, ela sente uma vibração discreta e o sistema avisa a central de segurança pela rede. A central acompanha a ocorrência, ouve o áudio gravado pelo microfone e recebe a localização enviada pelo Telegram. A usuária pode cancelar um alarme falso com uma senha.

O objetivo deste trabalho é desenvolver e validar esse sistema com um ESP32 que funcione como servidor na arquitetura cliente-servidor e use a pilha TCP/IP. Os clientes da rede devem conseguir:

a) consultar os dados dos sensores (botão e microfone) em tempo real;
b) enviar comandos ao atuador (motor de vibração) e ao sistema;
c) ver o endereço IP, o endereço MAC, o estado da conexão e o estado do alerta;
d) acessar um log persistente na memória flash, com eventos classificados como INFO, AVISO e ERRO, com data e hora.

O relatório segue esta ordem: a seção 2 traz a fundamentação teórica e os projetos semelhantes; a seção 3, a arquitetura e as regras de funcionamento; a seção 4, o desenvolvimento; a seção 5, os testes; e as seções 6 e 7, os resultados e a conclusão.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Sistemas ciberfísicos

Sistemas ciberfísicos integram computação, comunicação em rede e processos físicos: sensores medem o ambiente, um processador decide e atuadores agem sobre o mundo real, em um ciclo contínuo (LEE; SESHIA, 2017). No HELP ME, o botão e o microfone são os sensores, o ESP32 é a unidade de processamento e comunicação, e o motor de vibração é o atuador. A rede liga esse ciclo físico à central de segurança e à usuária.

2.2 Modelo OSI e modelo TCP/IP

O modelo OSI organiza a comunicação em sete camadas, da física à de aplicação (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1994). A Internet usa o modelo TCP/IP, que agrupa essas funções em quatro camadas: enlace, internet, transporte e aplicação (KUROSE; ROSS, 2021). O Quadro 1 mostra como o HELP ME usa cada camada.

Quadro 1 – Camadas do modelo TCP/IP no HELP ME

Camada TCP/IP	Camadas OSI	No HELP ME
Aplicação	Aplicação, apresentação e sessão	HTTP (central e rotas), HTTPS (Telegram), NTP (hora), JSON
Transporte	Transporte	TCP na porta 80 (servidor) e 443 (Telegram); UDP na porta 123 (NTP)
Internet	Rede	IPv4: o ESP32 recebe um endereço IP do roteador (DHCP)
Enlace	Enlace e física	Wi-Fi 802.11 b/g/n em 2,4 GHz, identificado pelo endereço MAC

Fonte: Os autores (2026).

2.3 Arquitetura cliente-servidor

Na arquitetura cliente-servidor, o servidor fica sempre ligado, em um endereço conhecido, esperando pedidos; os clientes iniciam a comunicação quando precisam (KUROSE; ROSS, 2021). No HELP ME, o ESP32 é o servidor: abre a porta 80, fica em escuta e responde aos navegadores da central e a ferramentas como o curl. Para falar com o Telegram, o ESP32 também age como cliente, fazendo pedidos HTTPS à API do Telegram (TELEGRAM, [20--]). Esse papel de cliente é um canal extra; todas as funções exigidas pelo TDE são atendidas pelo servidor da porta 80.

2.4 TCP e UDP

O TCP é orientado à conexão: estabelece a conexão com o aperto de mão em três vias (SYN, SYN-ACK, ACK), numera os dados, confirma o recebimento e retransmite o que se perdeu, garantindo entrega completa e em ordem (EDDY, 2022). O UDP envia datagramas sem conexão e sem confirmação, com menos atraso, mas sem garantia de entrega (POSTEL, 1980). O HELP ME usa TCP para tudo o que não pode se perder, como o alerta, os comandos e os dados da central, e o UDP aparece apenas na sincronização da hora por NTP, em que um pacote perdido é simplesmente pedido de novo.

2.5 Sockets

Um socket é o ponto de comunicação entre um programa e a rede, identificado pelo endereço IP e pela porta (TANENBAUM; WETHERALL, 2011). O servidor cria um socket, associa-o a uma porta (bind), passa a escutar (listen) e aceita cada conexão que chega (accept). No ESP32, a biblioteca WebServer faz essas etapas sobre a pilha lwIP: o socket fica escutando na porta 80, e a cada volta do programa principal a função handleClient aceita a conexão pendente, lê o pedido e envia a resposta (ESPRESSIF SYSTEMS, [20--]).

2.6 Protocolo HTTP

O HTTP é um protocolo de pedido e resposta. O cliente envia um método (como GET) e um caminho (como /status), e o servidor devolve um código de estado (200 quando dá certo, 404 quando a rota não existe) e o conteúdo (FIELDING; NOTTINGHAM; RESCHKE, 2022). O ESP32 responde em HTML para a página da central e em JSON para os dados, o que permite usar tanto o navegador quanto o curl. Para o Telegram, o HTTP é transportado dentro de uma conexão TLS (HTTPS), que cifra os dados.

2.7 Projetos semelhantes

Foram estudados seis projetos com diagrama de ligação, código e bibliotecas publicados (Quadro 2). Os dois primeiros são dispositivos de segurança pessoal; os outros quatro mostram, cada um, uma parte técnica usada no HELP ME.

Quadro 2 – Projetos semelhantes

Projeto	Diagrama, código e bibliotecas	O que foi aproveitado	Diferença do HELP ME
Smart SOS Device with Emergency Alert Button and Alarm (VASOYA, 2022)	Diagrama do circuito e de ligação; código Arduino; TinyGPS++	Botão de pânico como sensor principal	O alerta vai para uma central na rede, e não por ligação ou SMS
Smart Safety Device (CHOUDHARY, [20--])	Tabela de ligações e fotos; código ESP32; WiFi, WiFiClientSecure, UniversalTelegramBot, TinyGPS++	Alerta pelo Telegram com link do mapa	Telegram com cancelamento por senha e coação; o ESP32 também é servidor
Telegram: ESP32 Motion Detection with Notifications (SANTOS, [20--]b)	Diagrama de ligação; código completo; WiFiClientSecure, UniversalTelegramBot, ArduinoJson	Envio de mensagens do ESP32 para o Telegram	Comunicação nos dois sentidos, com comandos e conversa
ESP32 Web Server – Arduino IDE (SANTOS, [20--]a)	Esquema elétrico; código completo; WiFi.h	Servidor HTTP na porta 80 controlando saídas	Rotas em JSON, log, histórico e central completa
ESP32 with INMP441 I2S Microphone (SANTOS, 2026)	Diagrama de ligação; três códigos; driver I2S do ESP-IDF	Leitura do INMP441 e cálculo do volume	Gravação em WAV na flash e reprodução na central
ESP32 Datalogger: Download Data File via Web Server (SANTOS, 2024)	Diagrama do circuito; código completo; ESPAsyncWebServer, SD, time.h	Registro com data e hora por NTP servido pela web	Log em LittleFS com níveis INFO, AVISO e ERRO

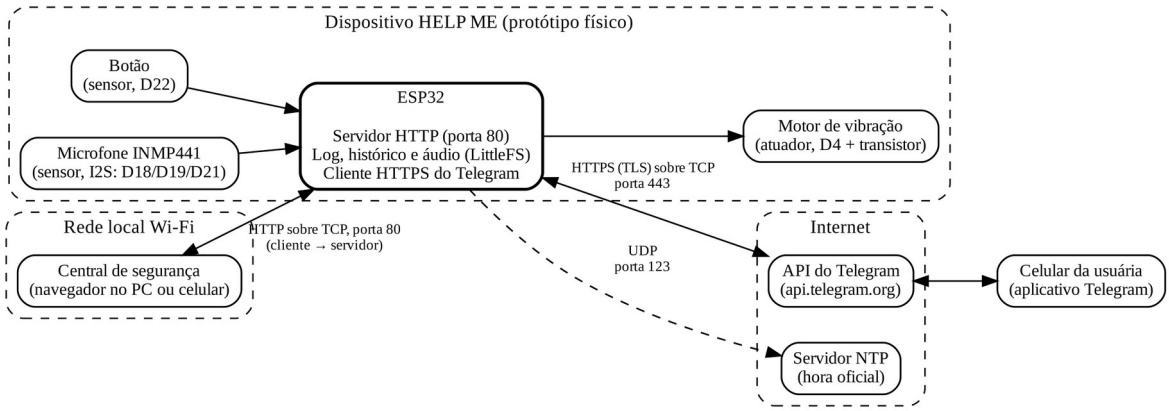
Fonte: Os autores (2026).

3 ARQUITETURA DO SISTEMA

3.1 Visão geral

O sistema tem três partes: o dispositivo (ESP32 com sensores e atuador), a central de segurança (navegador na rede local) e o canal da usuária (Telegram). A Figura 1 mostra como elas se ligam e quais protocolos e portas cada ligação usa.

Figura 1 – Arquitetura do HELP ME



Fonte: Os autores (2026).

O ESP32 conecta-se ao Wi-Fi no modo estação, recebe um IP e abre o servidor HTTP na porta 80. A central é um cliente: o navegador pede a página ao ESP32 e depois consulta as rotas de dados a cada segundo. A usuária não acessa o servidor diretamente: ela conversa com um bot do Telegram, e o ESP32 busca e envia essas mensagens pela API do Telegram. A hora oficial vem de um servidor NTP e é usada no log e no histórico.

Dentro do ESP32, o trabalho é dividido em três tarefas que rodam ao mesmo tempo: o programa principal (servidor HTTP, botão, motor e regras), uma tarefa de áudio no núcleo 1 (leitura do microfone e gravação) e uma tarefa do Telegram no núcleo 0 (comunicação HTTPS). Assim, uma resposta lenta da Internet não trava o botão nem o servidor.

3.2 Componentes e ligações

O Quadro 3 mostra os componentes e os pinos usados. A pinagem completa está no Apêndice A.

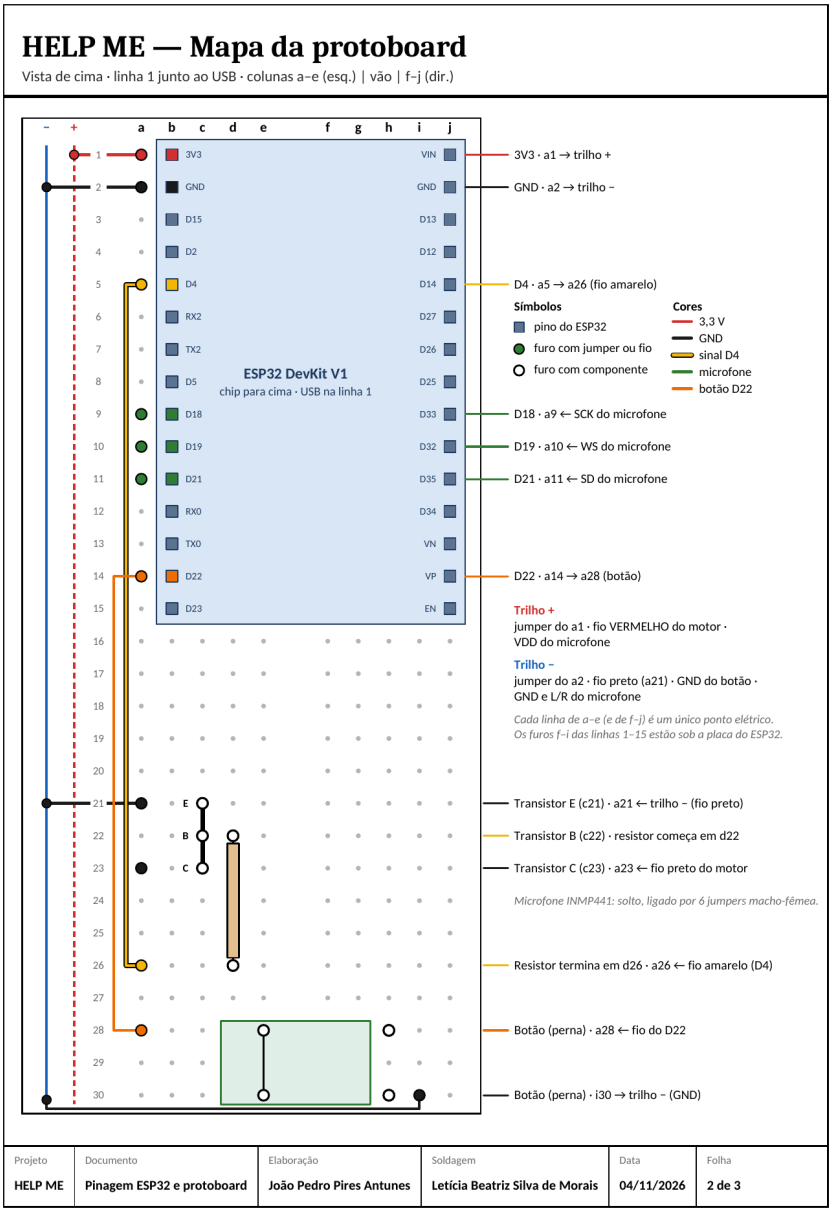
Quadro 3 – Componentes e pinos

Componente	Pino do ESP32	Função
ESP32 DevKit V1	—	Processa, conecta ao Wi-Fi e roda o servidor
Botão (INPUT_PULLUP)	D22	Sensor: aciona o alerta quando segurado por 1 s
Microfone digital INMP441	D18 (SCK), D19 (WS), D21 (SD)	Sensor: mede o volume e grava o áudio
Motor de vibração 3 V + transistor 2N2222A + resistor + diodo	D4	Atuador: vibra para confirmar o acionamento
Alimentação	3V3 e GND	Alimenta o microfone e o motor

Fonte: Os autores (2026).

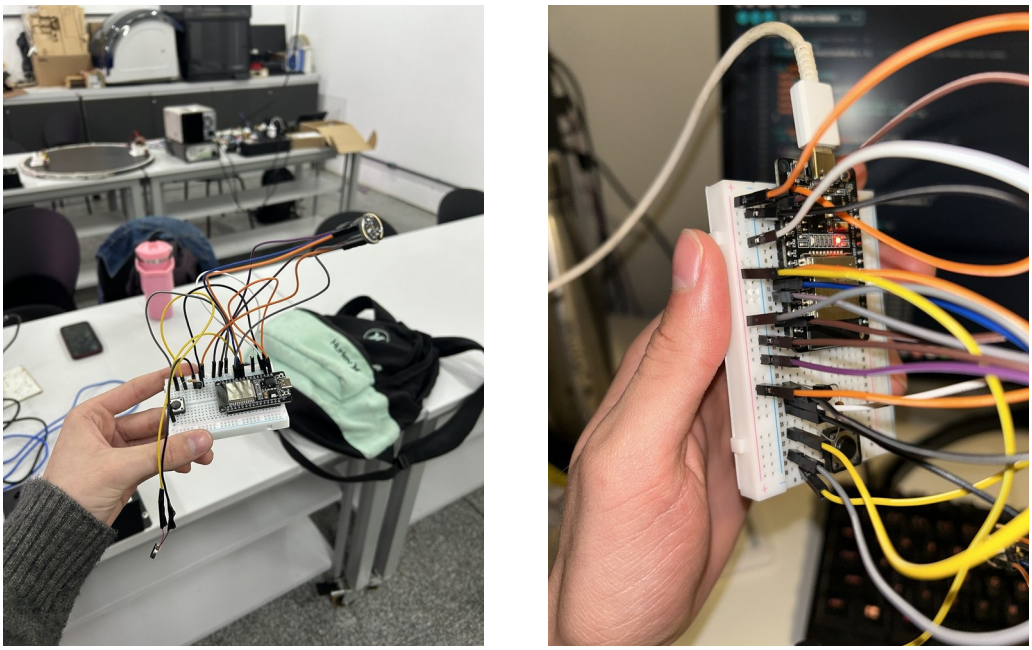
O motor não é ligado direto no pino: o pino D4 aciona a base do transistor, que liga o motor. Assim, o ESP32 não precisa fornecer a corrente do motor, e o diodo protege o circuito do pico de tensão quando o motor desliga.

Figura 2 – Mapa de ligações na protoboard



Fonte: Os autores (2026).

Figura 3 – Protótipo físico montado na protoboard, antes da case 3D

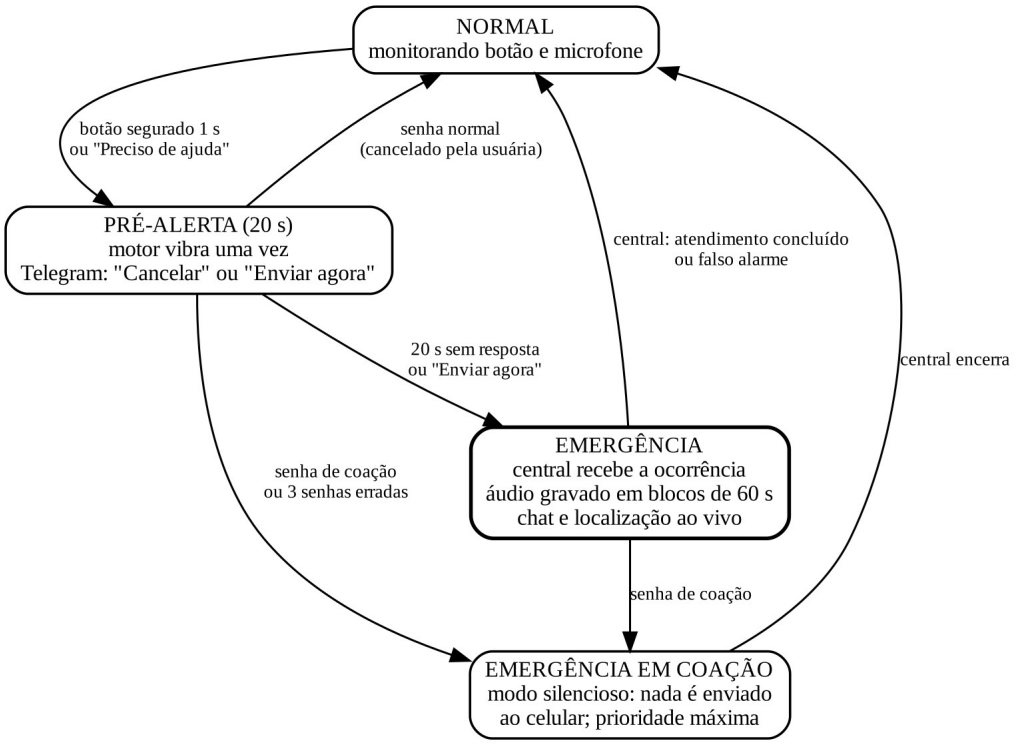


Fonte: Os autores (2026).

3.3 Regras de funcionamento

A Figura 4 resume os estados do sistema e o que leva de um estado a outro.

Figura 4 – Estados e regras de funcionamento



Fonte: Os autores (2026).

As regras foram pensadas para a situação real de uma vítima, em que o agressor pode estar perto e o celular pode estar nas mãos dele:

a) **acionamento:** o botão precisa ser segurado por 1 segundo; toques curtos são ignorados, o que evita disparos acidentais. O motor vibra uma única vez, apenas para confirmar o acionamento. A usuária também pode pedir ajuda pelo botão "Preciso de ajuda" no Telegram;
b) **pré-alerta:** depois do acionamento há 20 segundos para cancelar. O Telegram mostra os botões "Cancelar alerta" e "Enviar agora". Se nada for feito, a ocorrência é enviada à central;
c) **cancelamento por senha:** para cancelar é preciso digitar a senha numérica. Uma senha errada é avisada; três senhas erradas seguidas são tratadas como coação;
d) **senha de coação:** é uma segunda senha que mostra no celular a mesma mensagem de cancelamento, mas envia a ocorrência à central em modo silencioso e com prioridade máxima. Nesse modo, nada mais é enviado ao celular, nem pelo chat nem por avisos automáticos;
e) **emergência:** a central recebe a ocorrência com código, hora, origem e prioridade. O microfone grava o áudio em blocos de 60 segundos, salvos em WAV na flash, e a

central pode pedir mais 60 segundos. Se faltar espaço, os áudios mais antigos são apagados;

f) **som alto:** volume acima do limite por mais de 1,5 segundo durante um alerta sobe a prioridade para máxima. O som não dispara o alerta sozinho e nunca liga o motor;

g) **localização:** só é aceita durante um alerta. Pode ser uma posição fixa ou a localização em tempo real do Telegram, que a central vê em um mapa ao vivo. Quando a ocorrência termina, o trajeto é apagado;

h) **encerramento:** só a central encerra uma emergência, como "atendimento concluído" ou "falso alarme". Durante a emergência, a senha normal vira um pedido de encerramento para a central decidir. Em cancelamentos e falsos alarmes, o áudio é apagado;

i) **segurança do canal:** o bot só aceita mensagens do chat cadastrado da usuária; mensagens de outras contas são ignoradas e registradas no log;

j) **rede e log:** se o Wi-Fi cair, o ESP32 tenta reconectar a cada 10 segundos e registra a queda e a volta no log. Todo evento é gravado na flash com data, hora e nível (INFO, AVISO ou ERRO).

3.4 Rotas do servidor (API)

O servidor responde às rotas do Quadro 4, todas pelo método GET, na porta 80. As rotas de dados respondem em JSON, e uma rota inexistente responde com o código 404.

Quadro 4 – Rotas do servidor HTTP do ESP32

Rota	Resposta	O que faz
/	HTML	Página da central de segurança
/status	JSON	Estado do sistema, IP, MAC, rede, RSSI, tempo ligado, hora, alerta, Telegram e memória
/sensor	JSON	Volume do microfone, limite, som alto e estado do botão
/atuador?estado=on | off | pulso	JSON	Liga, desliga ou dá um pulso de 0,5 s no motor
/limite?valor=N	JSON	Ajusta o limite de som alto
/simular	JSON	Dispara um alerta de teste, como se o botão fosse apertado
/resolver | /falso	JSON	Encerra como atendimento concluído ou como falso alarme
/chat | /chat/enviar?texto=...	JSON	Lê a conversa e envia mensagem da central para a usuária
/audio?oc=CÓDIGO	WAV	Áudio gravado da ocorrência (também durante a gravação)
/gravar_mais	JSON	Grava mais 60 s de áudio
/localizacao?desde=N	JSON	Pontos novos da localização e status (ao vivo, parada, encerrada ou fixa)
/pedir_localizacao	JSON	Pede pelo Telegram a localização em tempo real (bloqueado em coação)
/historico | /estatisticas	JSON	Últimas ocorrências e números calculados do histórico

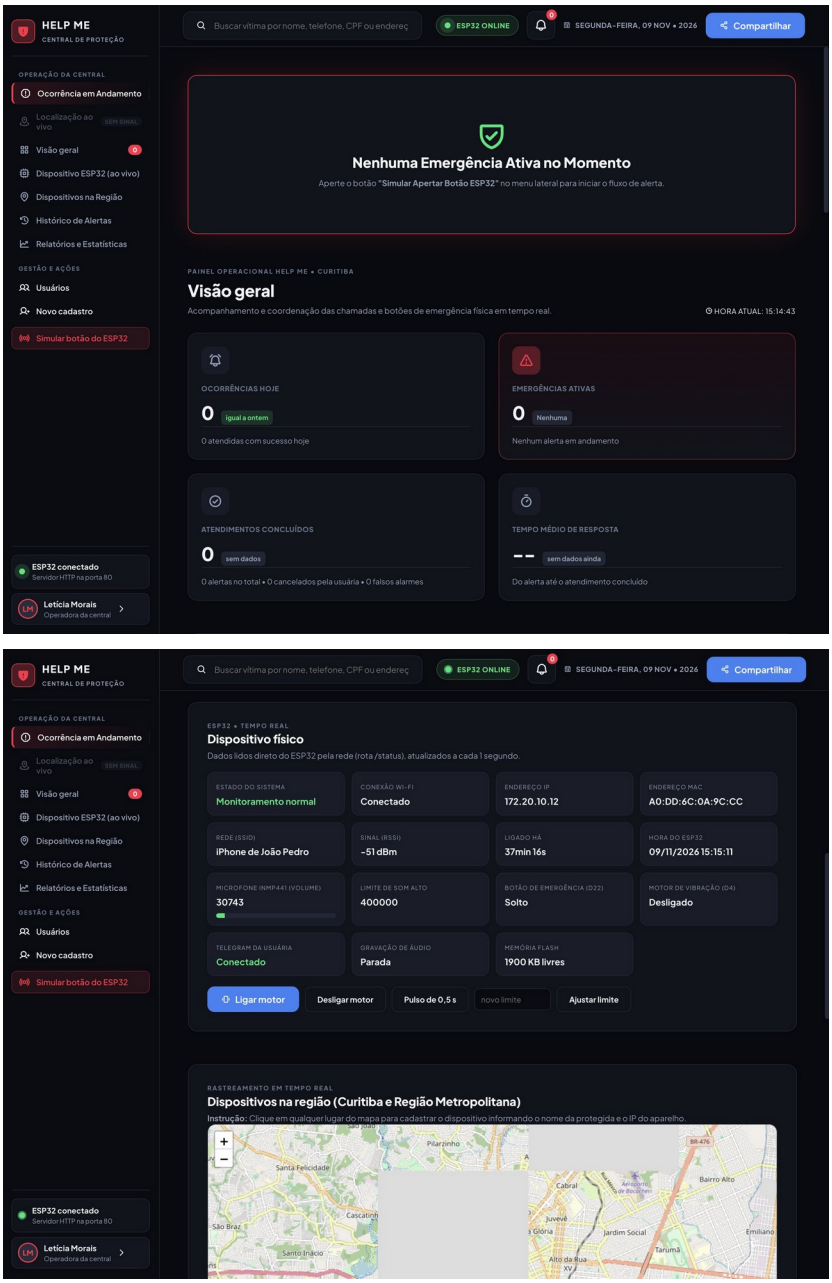
Rota	Resposta	O que faz
/log?ultimas=N | /log/limpar	Texto	Lê ou limpa o log da flash

Fonte: Os autores (2026).

3.5 Central de segurança (cliente web)

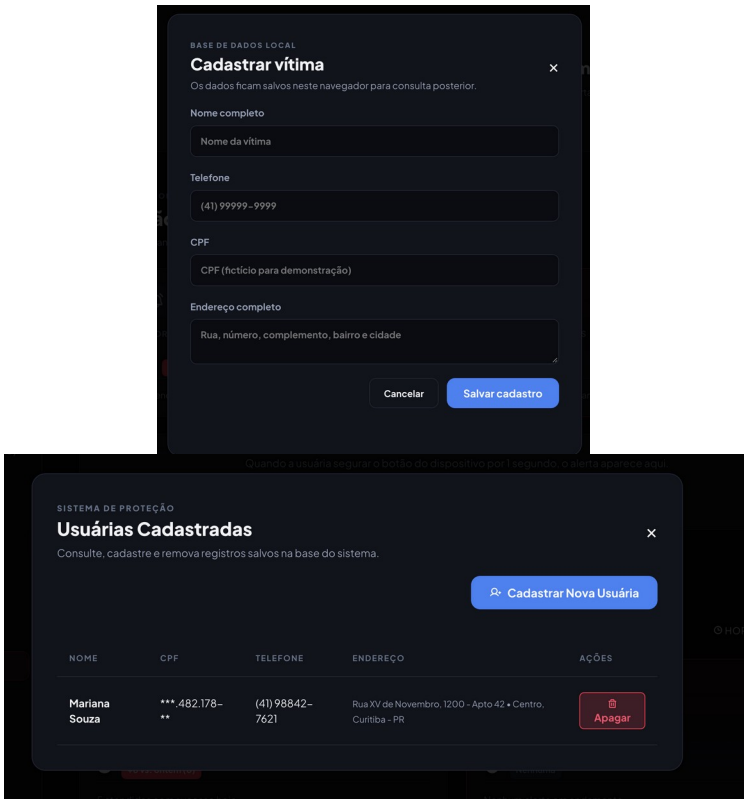
A central é a interface cliente do sistema. É uma página HTML, CSS e JavaScript servida pelo próprio ESP32 e aberta no navegador do computador ou do celular. Ela consulta a rota /status a cada segundo e mostra se o ESP32 está online, os dados do dispositivo ao vivo (IP, MAC, sinal, volume, botão e motor), a ocorrência em andamento com o player de áudio e o chat, o histórico e as estatísticas calculadas a partir das ocorrências reais gravadas no ESP32. A central também cadastra e lista as usuárias protegidas (Figura 6); os dados cadastrais da vítima são fictícios, por se tratar de um protótipo.

Figura 5 – Central de segurança: visão geral e painel do dispositivo físico



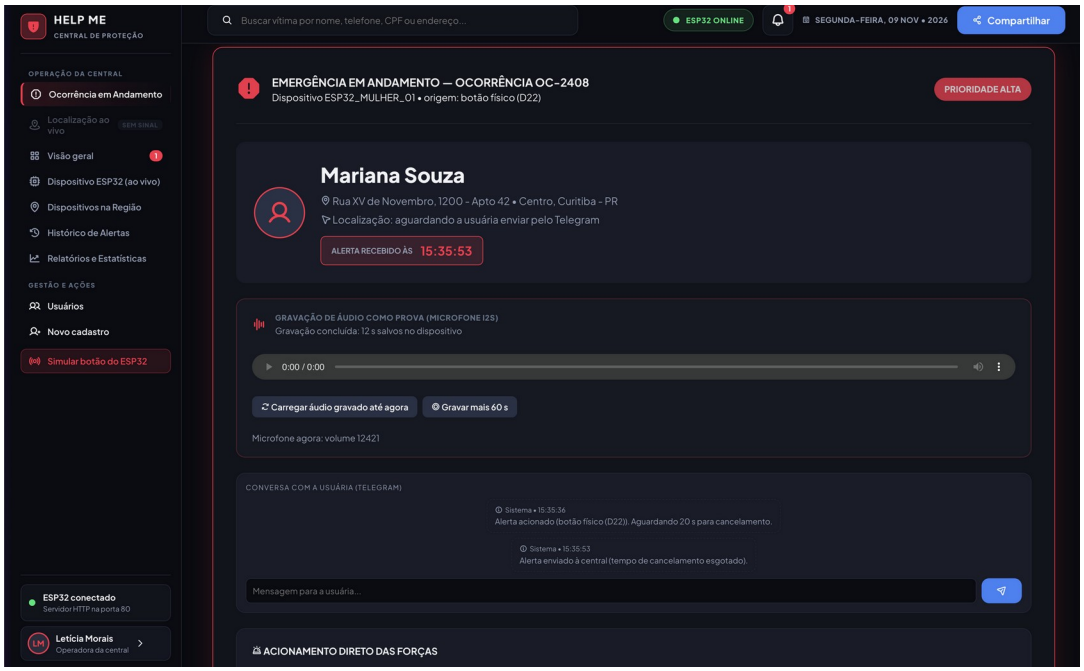
Fonte: Os autores (2026).

Figura 6 – Cadastro de usuárias na central



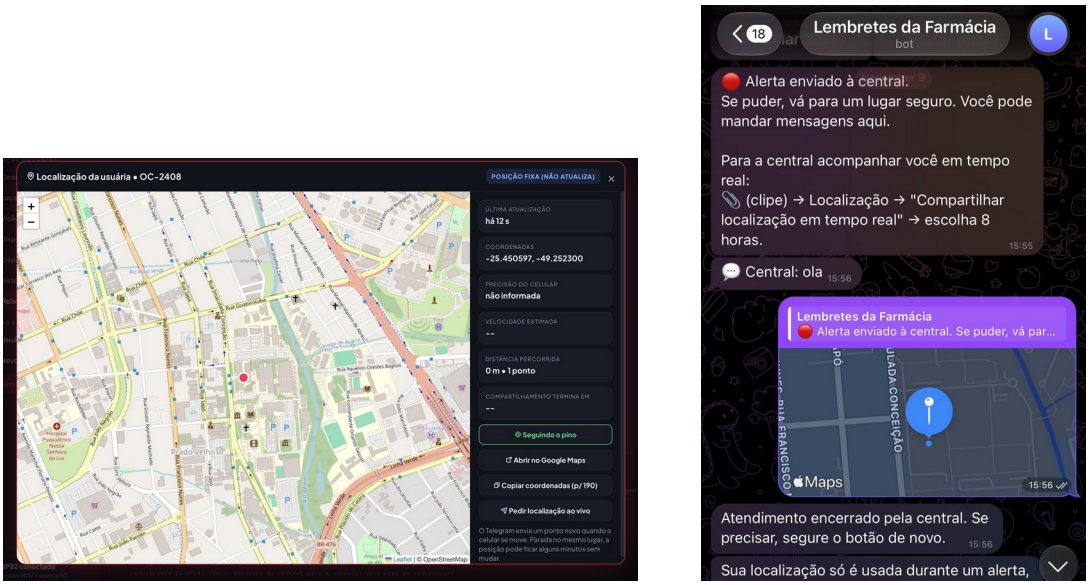
Fonte: Os autores (2026).

Figura 7 – Central com uma ocorrência em andamento



Fonte: Os autores (2026).

Figura 8 – Localização enviada pela usuária: mapa na central e confirmação no Telegram



Fonte: Os autores (2026).

3.6 Canal da usuária (Telegram)

A usuária usa apenas o Telegram, que já está no celular e não chama atenção. O bot aceita os comandos /start (menu), /status (estado do dispositivo), /teste (testa sem avisar a central) e /ajuda (telefones 190, 192 e 180), além dos botões "Enviar localização atual", "Estou bem" e "Preciso de ajuda". Durante o alerta, as mensagens que ela escreve aparecem no chat da central, e as respostas da central chegam a ela pelo bot.

Figura 9 – Telegram da usuária: comandos /status e /teste, pré-alerta e cancelamento com senha



Fonte: Os autores (2026).

4 DESENVOLVIMENTO

4.1 Equipe

Quadro 5 – Equipe

RA	Nome	Papel principal
40132879	Eduardo Hashimoto Yonegura	Hardware, protótipo, pesquisa de áudio, testes finais e vídeos
40126989	Gabriel André Moro	Pagamento das peças do projeto físico e do botão da case
40126364	Guilherme Sakagami Spitzner	Ajuda na montagem física do circuito
40126944	João Pedro Pires Antunes	Montagem do circuito, firmware, Telegram, software, testes finais, edição do vídeo, GitHub e versão final do relatório
40123207	Letícia Beatriz Silva de Morais	Soldagem, interface web, documentação, case 3D, revisão do relatório e slides

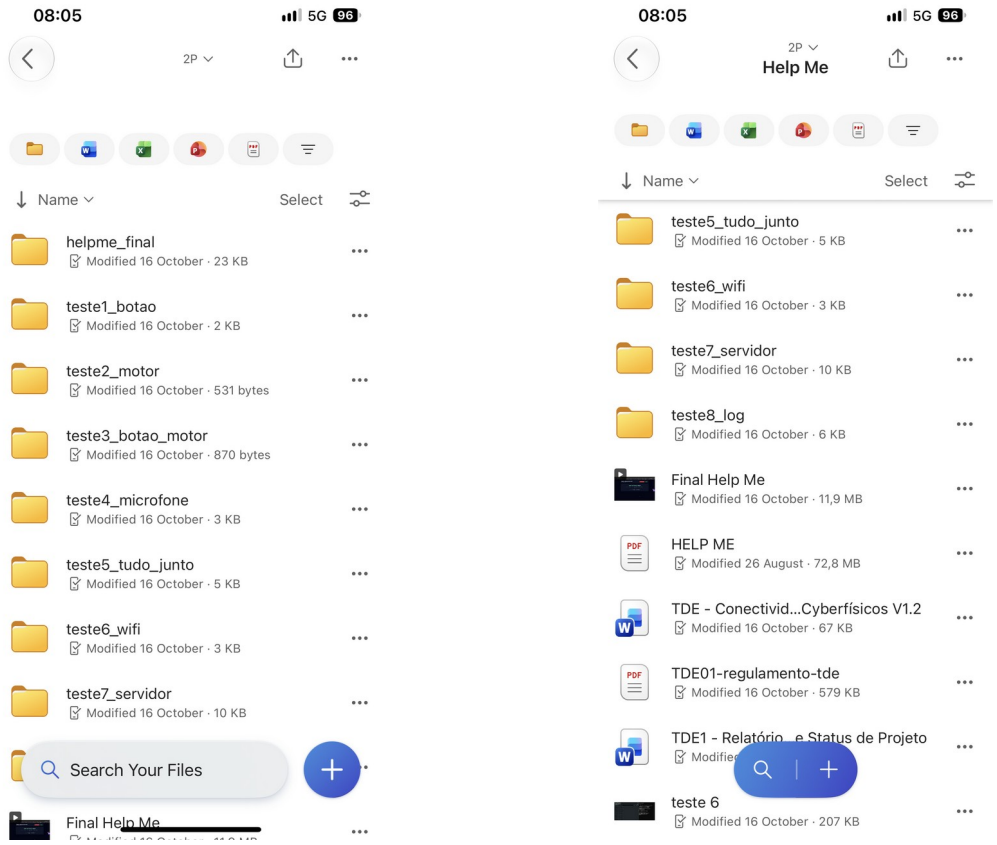
Fonte: Os autores (2026).

4.2 Etapas

09/10/2026 – montagem física. Participaram João Pedro, Letícia, Guilherme e Eduardo. João Pedro montou todo o circuito e a fiação, Letícia fez a soldagem, e Guilherme e Eduardo ajudaram na montagem. Gabriel não esteve presente.

15/10/2026 – testes das peças. João Pedro testou cada peça e cada parte do código separadamente (botão, motor, microfone, Wi-Fi, servidor e log), guardou cada código em uma pasta e registrou a maior parte dos prints e vídeos (Figura 10).

Figura 10 – Pastas com os códigos de teste



Fonte: Os autores (2026).

16/10/2026 – testes de rede e evolução do projeto. Participaram João Pedro, Letícia e Eduardo; Gabriel e Guilherme não estiveram presentes. Eduardo e João Pedro atualizaram o projeto no Wokwi. Letícia iniciou o design externo (case) e aprimorou a página web. João Pedro elaborou o documento de pinagem do ESP32 (Apêndice A). Eduardo iniciou a pesquisa sobre gravação de áudio no ESP32, e a ideia de gravar áudio como prova foi definida por Eduardo, João Pedro e Letícia. João Pedro e Eduardo realizaram os testes da primeira rodada e coletaram as evidências (seção 5.2).

04/11/2026 – versão final. João Pedro fez toda a parte de melhoria do funcionamento do projeto: melhorou as regras de funcionamento (seção 3.3), corrigiu pontos que estavam dando errado (seção 4.4), configurou o Telegram como canal da usuária e fez a versão final do código, com a nova central de segurança.

09/11/2026 – testes finais e preparação da apresentação. João Pedro e Eduardo fizeram os testes finais do sistema (rede, servidor, central web, Telegram, atuador e log) e deixaram tudo pronto para a apresentação. Os dois também gravaram os vídeos de demonstração, entre eles o vídeo geral do projeto que vai nos slides. Letícia revisou o relatório inteiro, corrigiu alguns pontos, devolveu o arquivo, comprou o botão para a case 3D e começou

a montar os slides. Gabriel não participou das atividades técnicas nem do relatório. Durante todo o projeto ele pagou as peças do projeto físico e, nesse dia, também o botão que Letícia pediu.

10/11/2026 – case 3D, vídeo, slides e relatório final. Letícia fez a case 3D do projeto com o botão comprado no dia anterior. João Pedro editou o vídeo geral do projeto e o enviou a Letícia, que o colocou nos slides finais da apresentação. Ela terminou os slides que começou em 09/11. João Pedro escreveu a versão final deste relatório.

4.3 Firmware

O firmware foi escrito em C++ na Arduino IDE 2, com o pacote de placas ESP32 da Espressif e o esquema de partição "No OTA (2MB APP/2MB SPIFFS)", que reserva 2 MB da flash para o sistema de arquivos LittleFS. O Quadro 6 lista as bibliotecas usadas.

Quadro 6 – Bibliotecas do firmware

Biblioteca	Uso no projeto
WiFi.h	Conexão Wi-Fi, IP, MAC e RSSI
WebServer.h	Servidor HTTP na porta 80 e rotas
LittleFS.h	Sistema de arquivos na flash: log, histórico e áudios
driver/i2s.h (ESP-IDF)	Leitura do microfone INMP441 pelo barramento I2S
WiFiClientSecure.h e HTTPClient.h	Pedidos HTTPS à API do Telegram
ArduinoJson (versão 7)	Leitura das respostas JSON do Telegram
time.h	Sincronização da hora por NTP (fuso de Brasília)
FreeRTOS (incluído no núcleo do ESP32)	Tarefas de áudio e do Telegram em paralelo, com filas protegidas por mutex

Fonte: Os autores (2026).

O trecho abaixo, copiado do firmware final, mostra duas funções ligadas aos requisitos do TDE: a gravação do log na flash, com data, hora e nível, que guarda o arquivo antigo quando ele passa do tamanho máximo, e a reconexão automática do Wi-Fi a cada 10 s, que registra a queda no log.

```
void registrar(const char* nivel, const String& msg) {
  String linha = agoraStr() + " [" + nivel + "] " + msg;
  Serial.println(linha);
  if (!fsOk) return;
  File f = LittleFS.open(LOG_FILE, "a");
  if (!f) return;
  f.println(linha);
  size_t tamanho = f.size();
  f.close();
  if (tamanho > LOG_MAX) {
    LittleFS.remove(LOG_OLD);
    LittleFS.rename(LOG_FILE, LOG_OLD);
  }
}

void verificarWiFi() {
  if (WiFi.status() == WL_CONNECTED) {
    if (!wifiEstavaConectado) aoConectar();
    return;
  }
  if (wifiEstavaConectado) {
    wifiEstavaConectado = false;
    registrar("AVISO", "Conexao Wi-Fi perdida. Tentando reconectar...");
    ultimaTentativa = millis();
  }
  if (millis() - ultimaTentativa >= RECONECTAR_MS) {
    ultimaTentativa = millis();
    Serial.println("Tentando reconectar ao Wi-Fi...");
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_SENHA);
  }
}
```

Fonte: Os autores (2026).

O código completo está no repositório público da equipe: https://github.com/jpiresantunes29-dotcom/HELP-ME. Lá ficam o firmware (src/esp32), os scripts de teste da central (src/client), este relatório, o roteiro de testes e as evidências (docs). No código publicado, o nome e a senha do Wi-Fi e o token do bot do Telegram foram trocados por valores genéricos, que precisam ser preenchidos antes de gravar o ESP32.

4.4 Problemas encontrados e soluções

Durante a integração da versão final no hardware, apareceram os problemas do Quadro 7, todos identificados pelo Monitor Serial.

Quadro 7 – Problemas encontrados e soluções

Problema	Causa	Solução
Erro "partition spiffs could not be found" e flash livre 0 KB	O esquema de partição escolhido não reservava espaço para o sistema de arquivos	Usar o esquema "No OTA (2MB APP/2MB SPIFFS)"
ESP32 reiniciando sem parar ("assert failed: xQueueSemaphoreTake")	O servidor e o microfone eram iniciados antes de a rede estar pronta	Nova ordem no setup: Wi-Fi, depois servidor, depois microfone e tarefas
Painel mostrando horários e números de exemplo	A interface nova vinha com dados de demonstração fixos	Relógio real, histórico e estatísticas calculados pelo ESP32

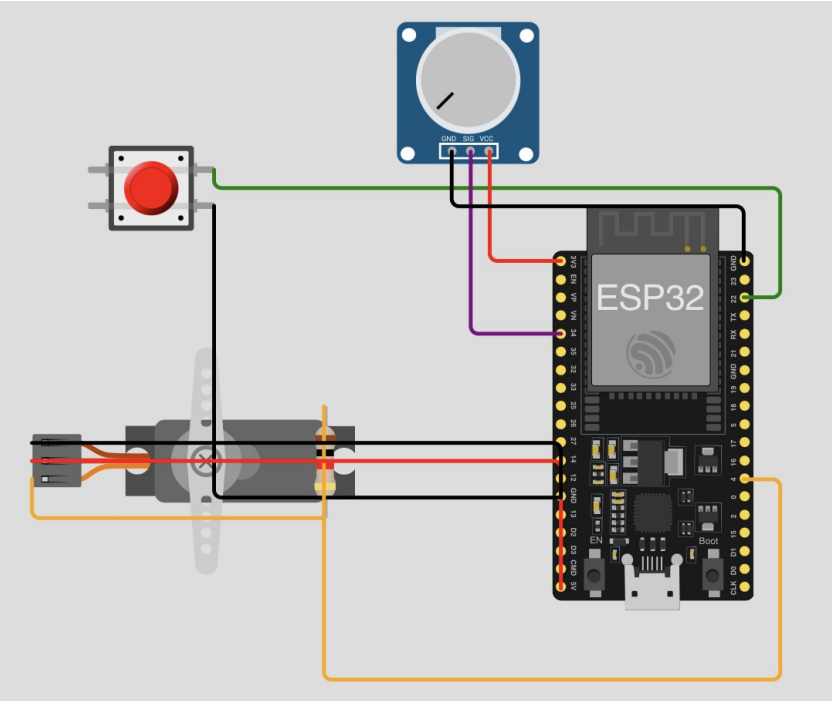
Problema	Causa	Solução
Alerta disparado logo após ligar ficava "sem hora"	A hora pela internet ainda não tinha sido sincronizada	A central usa a hora do computador até o ESP32 sincronizar

Fonte: Os autores (2026).

4.5 Simulação no Wokwi

Uma simulação no Wokwi foi usada apenas como apoio ao desenvolvimento. Como o simulador não tem o microfone nem o motor do projeto, foram usados um potenciômetro e um servo no lugar (Figura 11). A avaliação considera o protótipo físico, como exige o TDE.

Figura 11 – Simulação no Wokwi



Fonte: Os autores (2026).

5 TESTES E VALIDAÇÃO

5.1 Ambiente de testes

Os testes foram feitos em duas rodadas. A primeira, em 16/10/2026, validou a rede e o servidor da segunda versão, com o ESP32 no ponto de acesso de um celular (IP 172.20.10.12) e na rede de casa (IP 192.168.18.23). A segunda, em 09/11/2026, com a versão final, validou as novas regras, o Telegram e a central, com o ESP32 no ponto de acesso do iPhone de João Pedro (IP 172.20.10.12, MAC A0:DD:6C:0A:9C:CC) e o computador de teste na mesma rede (IP 172.20.10.14). O IP muda de uma rede para outra porque é entregue pelo roteador (DHCP).

Ferramentas usadas: Monitor Serial da Arduino IDE (115200 baud), ping e Test-NetConnection no Windows (primeira rodada), ping, nc e curl no Terminal do macOS (segunda rodada), navegador, um script de PowerShell com 100 requisições seguidas e o Telegram no celular.

5.2 Primeira rodada: rede e servidor (16/10/2026)

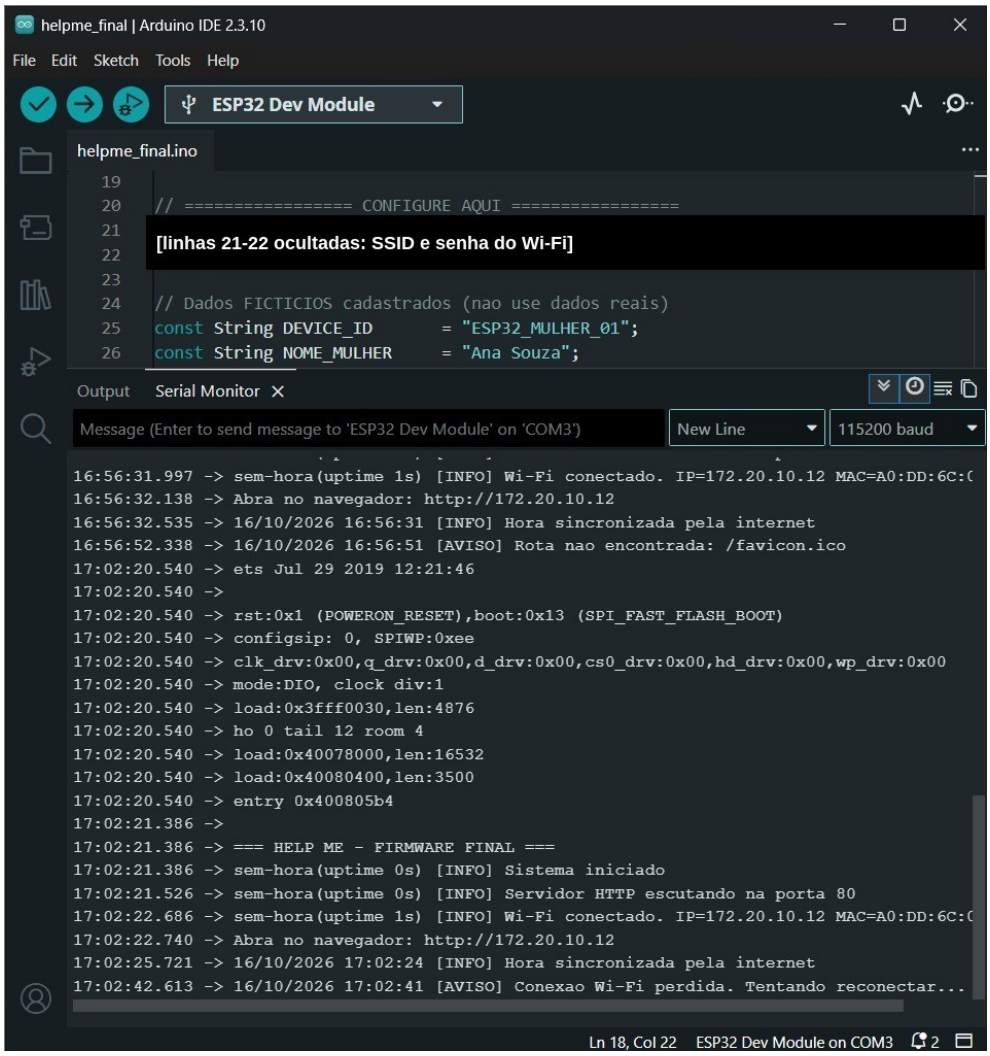
O Quadro 8 resume os resultados, comprovados pelas figuras seguintes.

Quadro 8 – Resultados da primeira rodada

Teste	Como foi feito	Resultado
Conectividade	ping 172.20.10.12	4 de 4 respostas, 0% de perda
Acesso à porta	Test-NetConnection na porta 80	TcpTestSucceeded: True
Cliente-servidor	Rotas / e /sensor no navegador	Respostas corretas em texto e JSON
Alerta e log	Botão, painel e som alto	Alertas registrados com a origem
Queda do Wi-Fi	Desligar e religar o ponto de acesso	Reconectou sozinho e registrou no log
Persistência	Reiniciar o ESP32 e abrir /log	Eventos mantidos na flash
Estresse	100 requisições seguidas	100 respostas com código 200
Acesso pelo celular	Painel e /status no iPhone	Funcionou

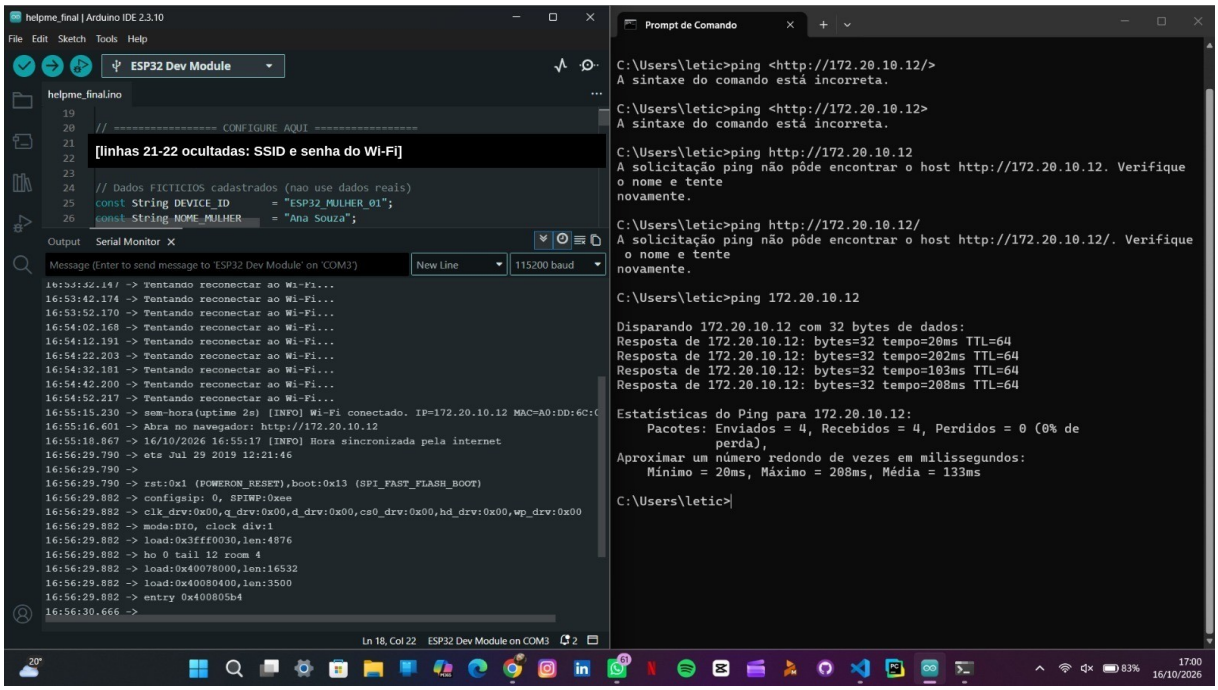
Fonte: Os autores (2026).

Figura 12 – ESP32 iniciando o servidor na porta 80



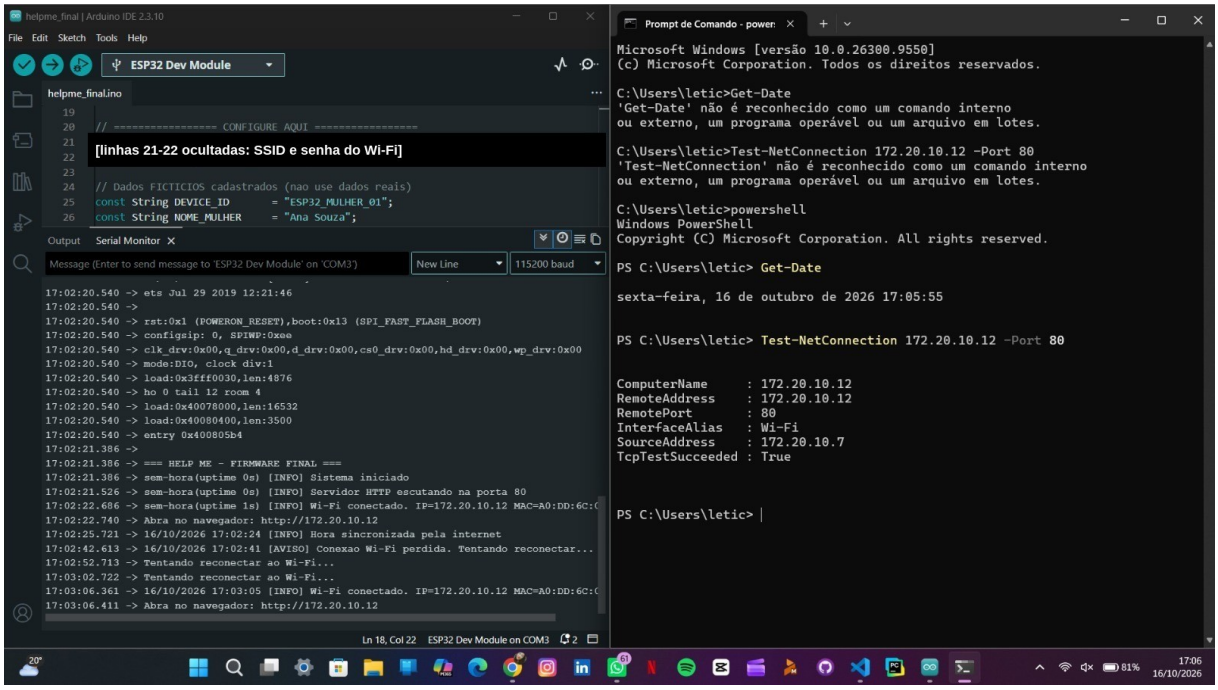
Fonte: Os autores (2026).

Figura 13 – Teste de ping



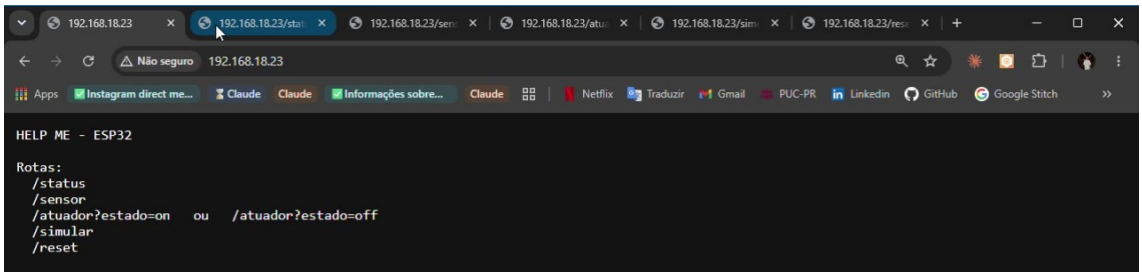
Fonte: Os autores (2026).

Figura 14 – Teste de acesso à porta 80

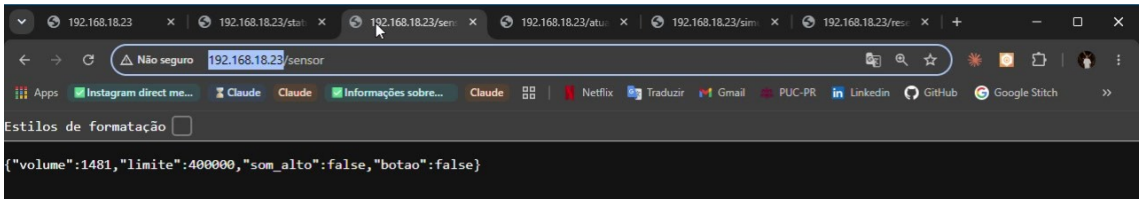


Fonte: Os autores (2026).

Figura 15 – Rota raiz e rota /sensor no navegador

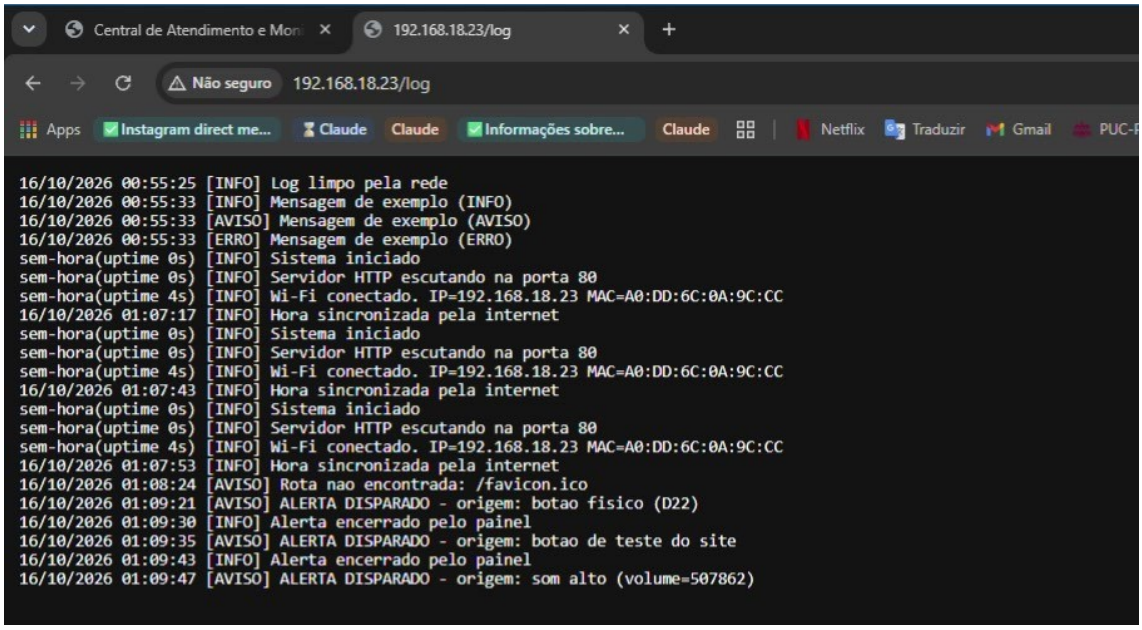


Fonte: Os autores (2026).



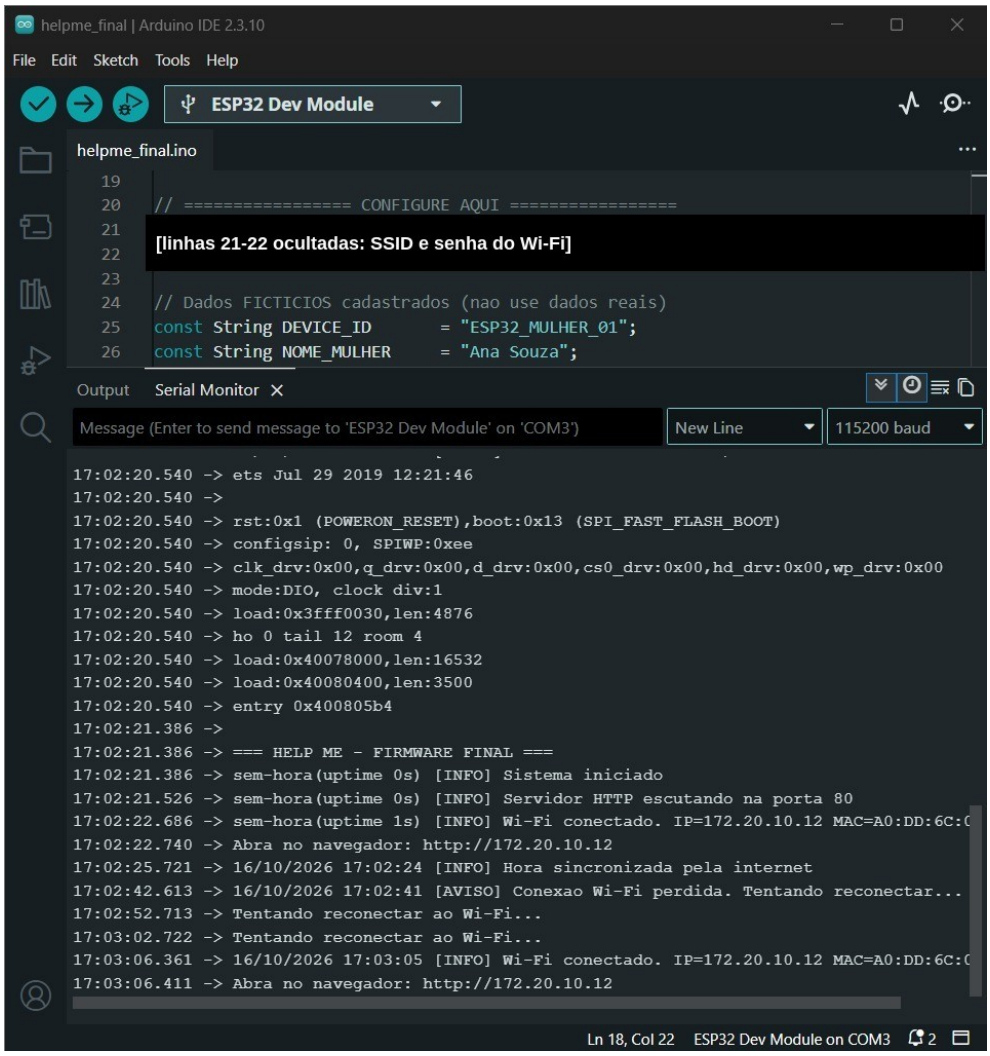
Fonte: Os autores (2026).

Figura 16 – Log com os alertas disparados



Fonte: Os autores (2026).

Figura 17 – Queda e reconexão do Wi-Fi



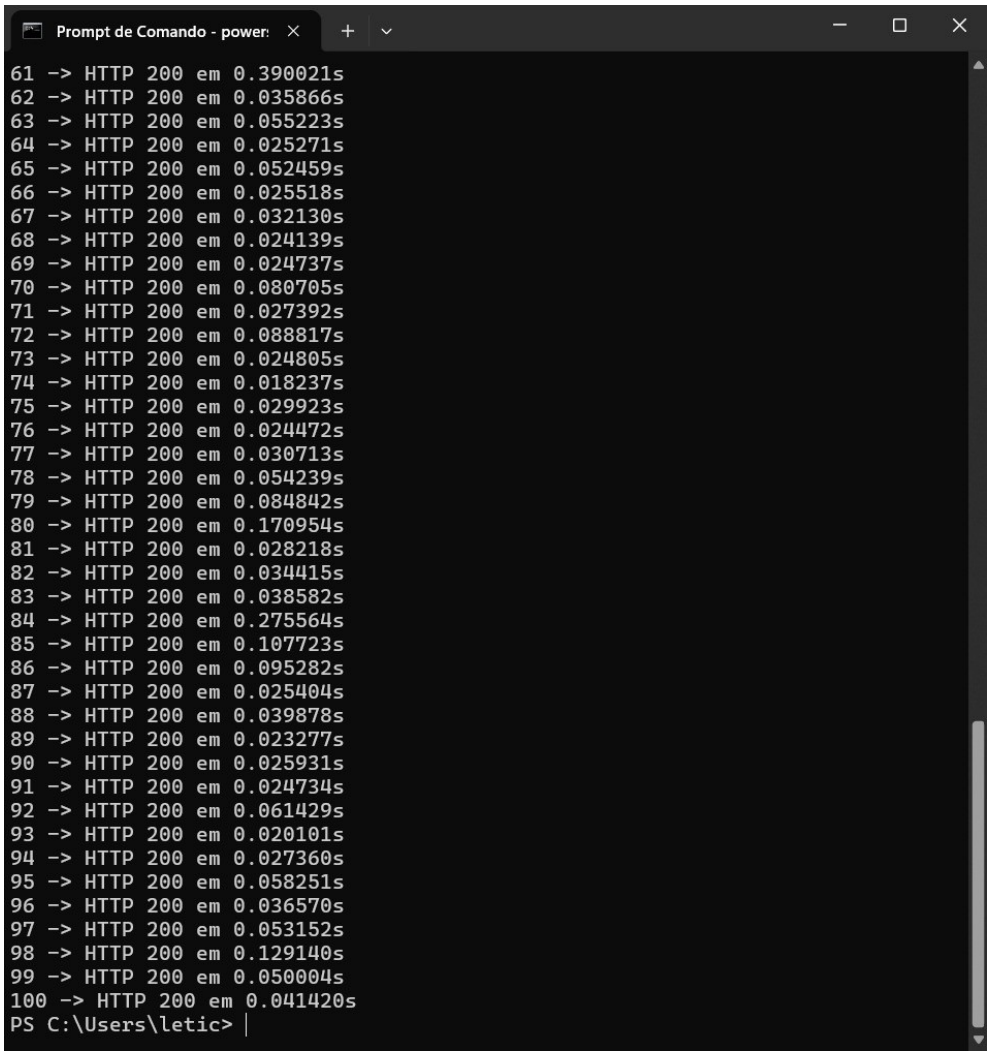
Fonte: Os autores (2026).

Figura 18 – Log mantido depois de reiniciar

```
16/10/2026 16:55:17 [INFO] Hora sincronizada pela internet
sem-hora(uptime 0s) [INFO] Sistema iniciado
sem-hora(uptime 0s) [INFO] Servidor HTTP escutando na porta 80
sem-hora(uptime 1s) [INFO] Wi-Fi conectado. IP=172.20.10.12 MAC=A0:DD:6C:0A:9C:CC
16/10/2026 16:56:31 [INFO] Hora sincronizada pela internet
16/10/2026 16:56:51 [AVISO] Rota nao encontrada: /favicon.ico
sem-hora(uptime 0s) [INFO] Sistema iniciado
sem-hora(uptime 0s) [INFO] Servidor HTTP escutando na porta 80
sem-hora(uptime 1s) [INFO] Wi-Fi conectado. IP=172.20.10.12 MAC=A0:DD:6C:0A:9C:CC
16/10/2026 17:02:24 [INFO] Hora sincronizada pela internet
16/10/2026 17:02:41 [AVISO] Conexao Wi-Fi perdida. Tentando reconectar...
16/10/2026 17:03:05 [INFO] Wi-Fi conectado. IP=172.20.10.12 MAC=A0:DD:6C:0A:9C:CC
sem-hora(uptime 0s) [INFO] Sistema iniciado
sem-hora(uptime 0s) [INFO] Servidor HTTP escutando na porta 80
sem-hora(uptime 1s) [INFO] Wi-Fi conectado. IP=172.20.10.12 MAC=A0:DD:6C:0A:9C:CC
16/10/2026 17:06:58 [INFO] Hora sincronizada pela internet
16/10/2026 17:07:29 [AVISO] Conexao Wi-Fi perdida. Tentando reconectar...
16/10/2026 17:08:51 [INFO] Wi-Fi conectado. IP=172.20.10.12 MAC=A0:DD:6C:0A:9C:CC
```

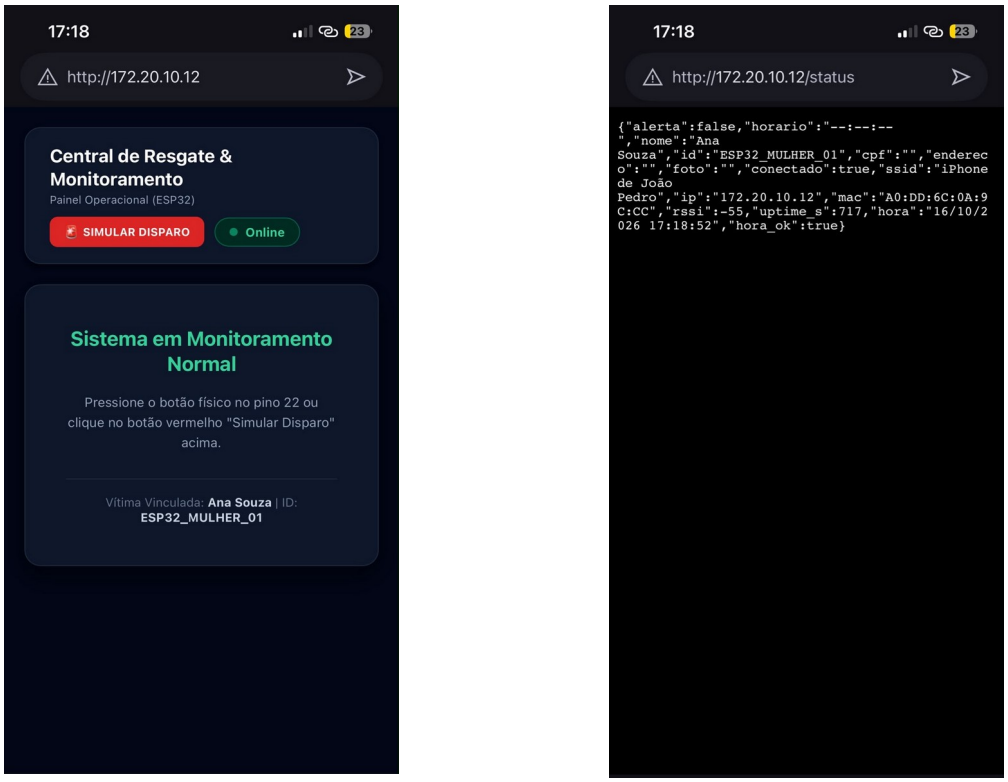
Fonte: Os autores (2026).

Figura 19 – Teste de estresse com 100 requisições



Fonte: Os autores (2026).

Figura 20 – Acesso pelo celular



Fonte: Os autores (2026).

5.3 Segunda rodada: versão final

A segunda rodada segue o roteiro de testes da equipe. O Quadro 9 mostra o resultado esperado de cada teste e o resultado obtido.

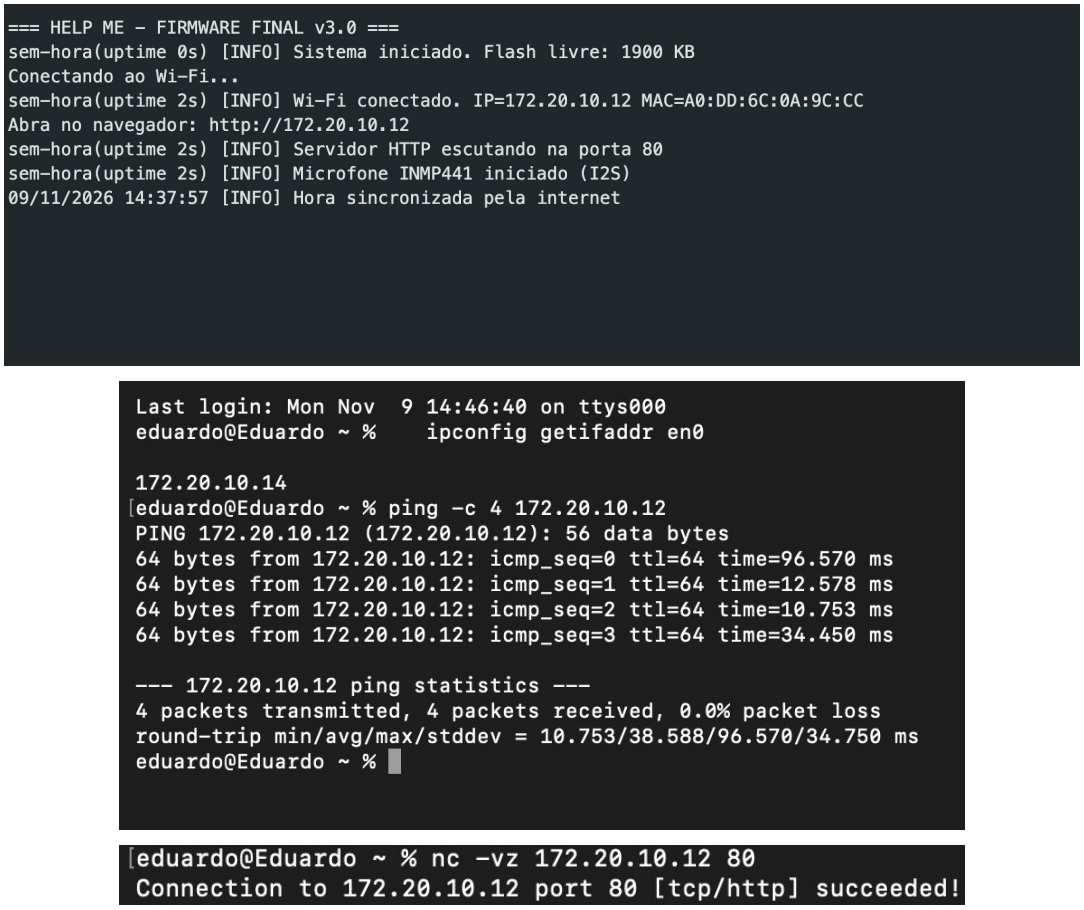
Quadro 9 – Resultados da segunda rodada

Teste (roteiro)	Resultado esperado	Obtido
Inicialização (A4)	Wi-Fi, servidor, microfone e hora sem reiniciar	OK (Figura 21)
Conectividade e porta (B1, B2)	Ping sem perda e porta 80 aberta	OK (Figura 21)
Comunicação cliente-servidor com curl (B3, B6)	JSON em /status, /sensor, /historico e /estatisticas; 404 em rota inexistente	OK (Figura 22)
Atuador pela rede (B5)	Motor liga e desliga com /atuador	OK (Figuras 23 e 5)
Microfone ao vivo na central (C3)	Barra de volume acompanha o som	OK (Figura 5)
Disparo pelo botão (E1, E2)	Toque curto ignorado; 1 s vibra e abre o pré-alerta	OK (Figuras 24 e 27)
Cancelamento com senha (E3, E4)	Senha certa cancela; senha errada é avisada	OK (Figuras 9 e 27)
Atendimento (F3, F4, F7)	Chat nos dois sentidos e áudio tocando na central	OK (Figuras 25 e 7; áudio e chat na central no vídeo, Figura 28)

Teste (roteiro)	Resultado esperado	Obtido
Localização ao vivo (G1 a G8)	Pino se move no mapa; status muda quando ela para	OK (Figura 8; posição no mapa da central no vídeo, Figura 28)
Coação (H1 a H4)	Celular mostra "cancelado"; central recebe em silêncio	OK (testado em 09/11/2026; celular na Figura 26)
Histórico e estatísticas (I1 a I7)	Desfechos reais, números corretos e trajeto apagado	OK (Figura 27)
Falhas (J1 a J5)	Reconexão do Wi-Fi e dados mantidos após reiniciar	OK (Figuras 17 e 18; repetido na versão final em 09/11/2026)

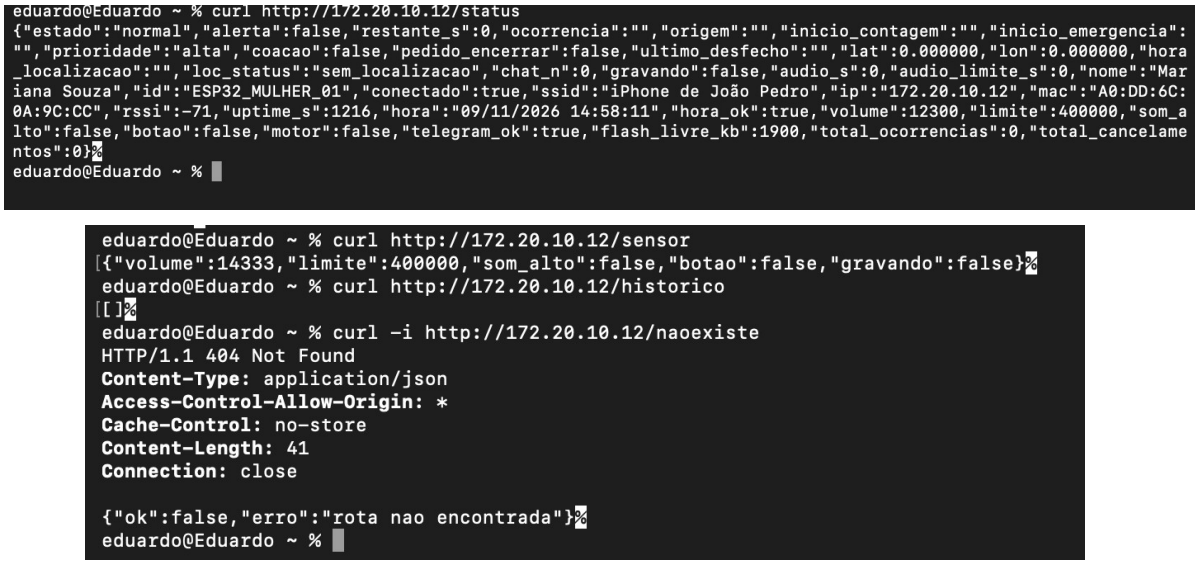
Fonte: Os autores (2026).

Figura 21 – Inicialização, ping e acesso à porta 80 na versão final



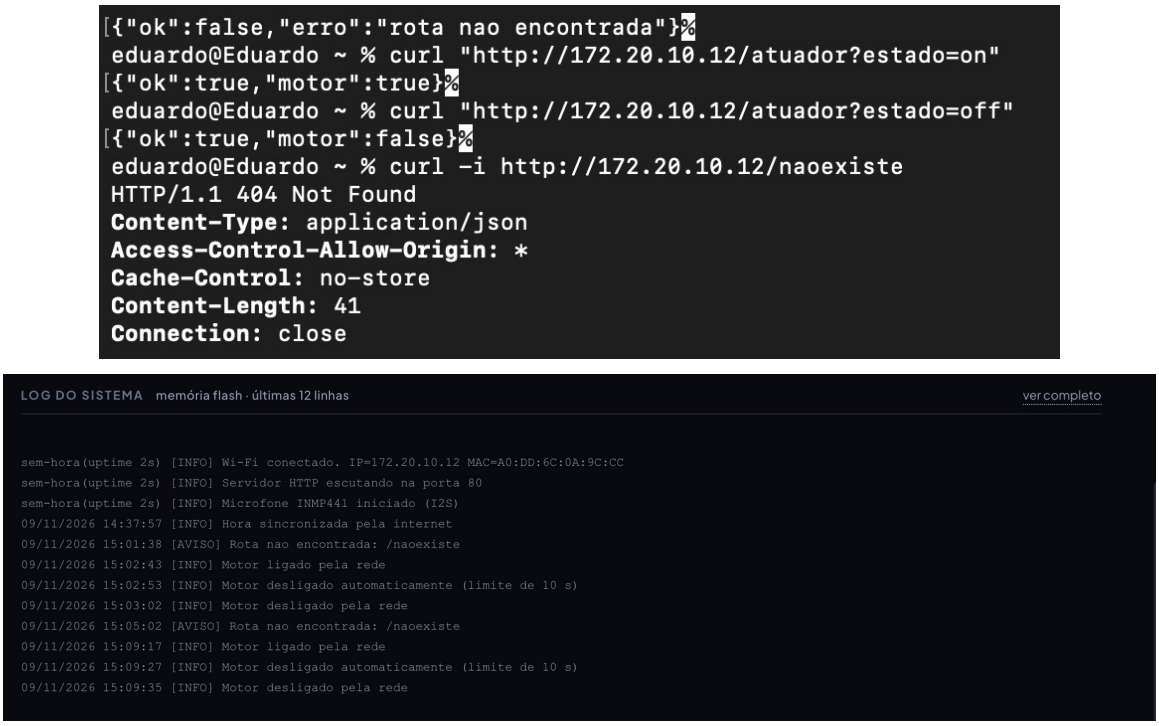
Fonte: Os autores (2026).

Figura 22 – Comunicação cliente-servidor com curl



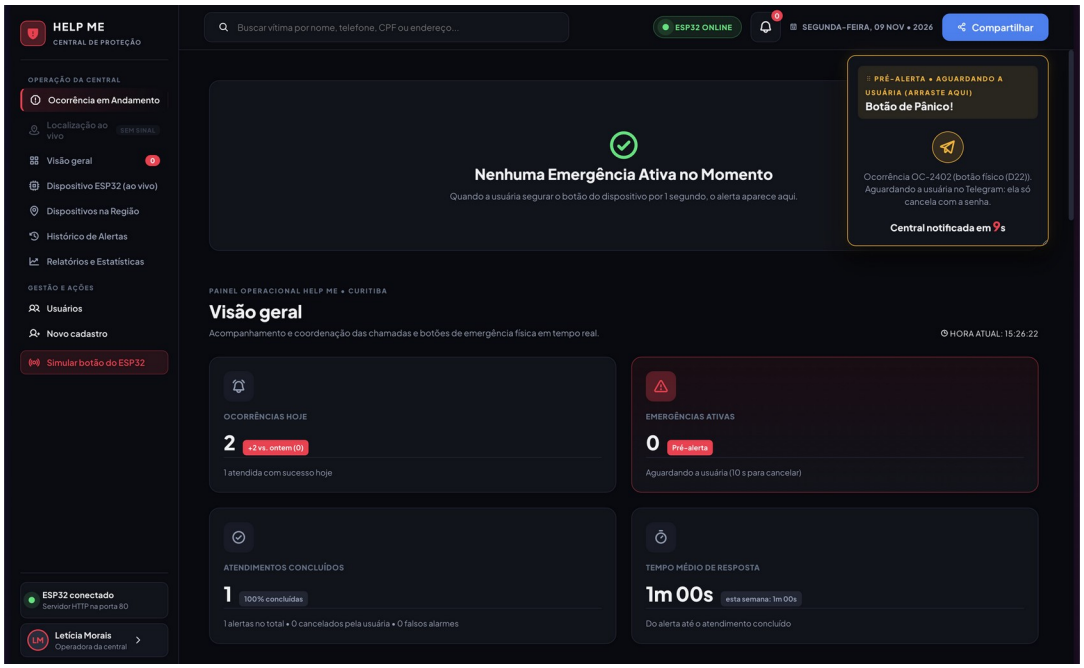
Fonte: Os autores (2026).

Figura 23 – Motor acionado pela rede e registro no log



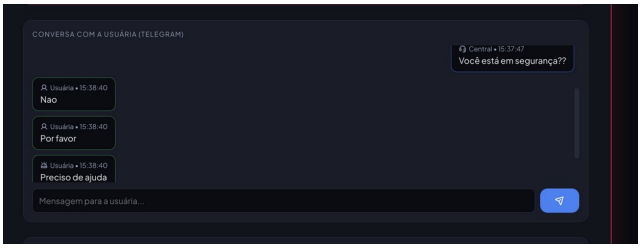
Fonte: Os autores (2026).

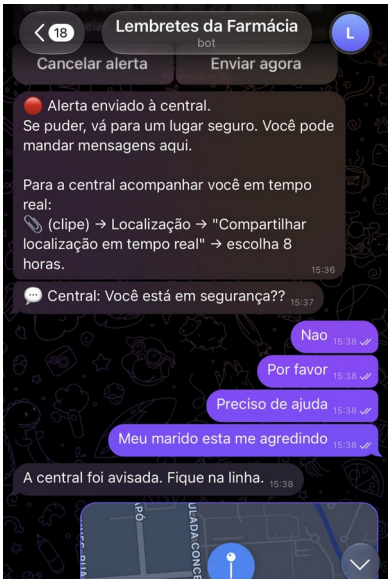
Figura 24 – Pré-alerta na central com contagem regressiva para o cancelamento



Fonte: Os autores (2026).

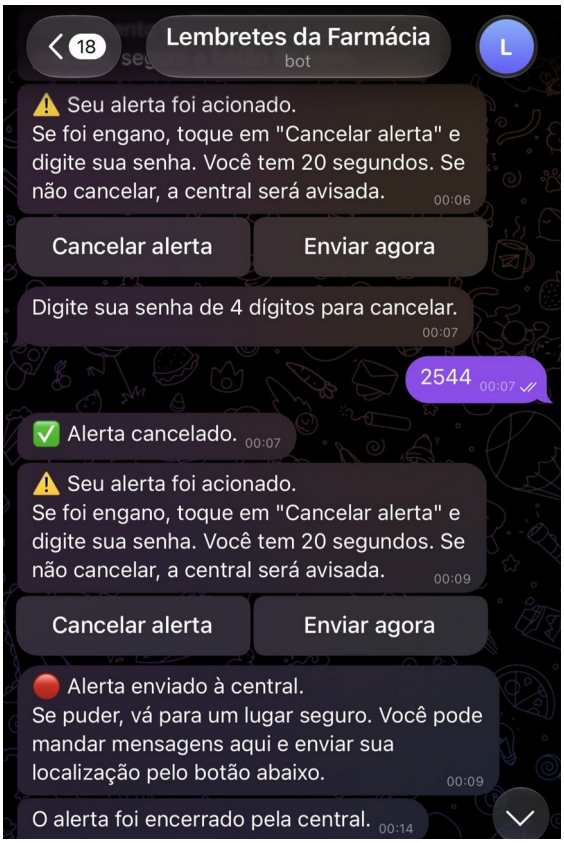
Figura 25 – Atendimento: conversa vista na central e no Telegram da usuária





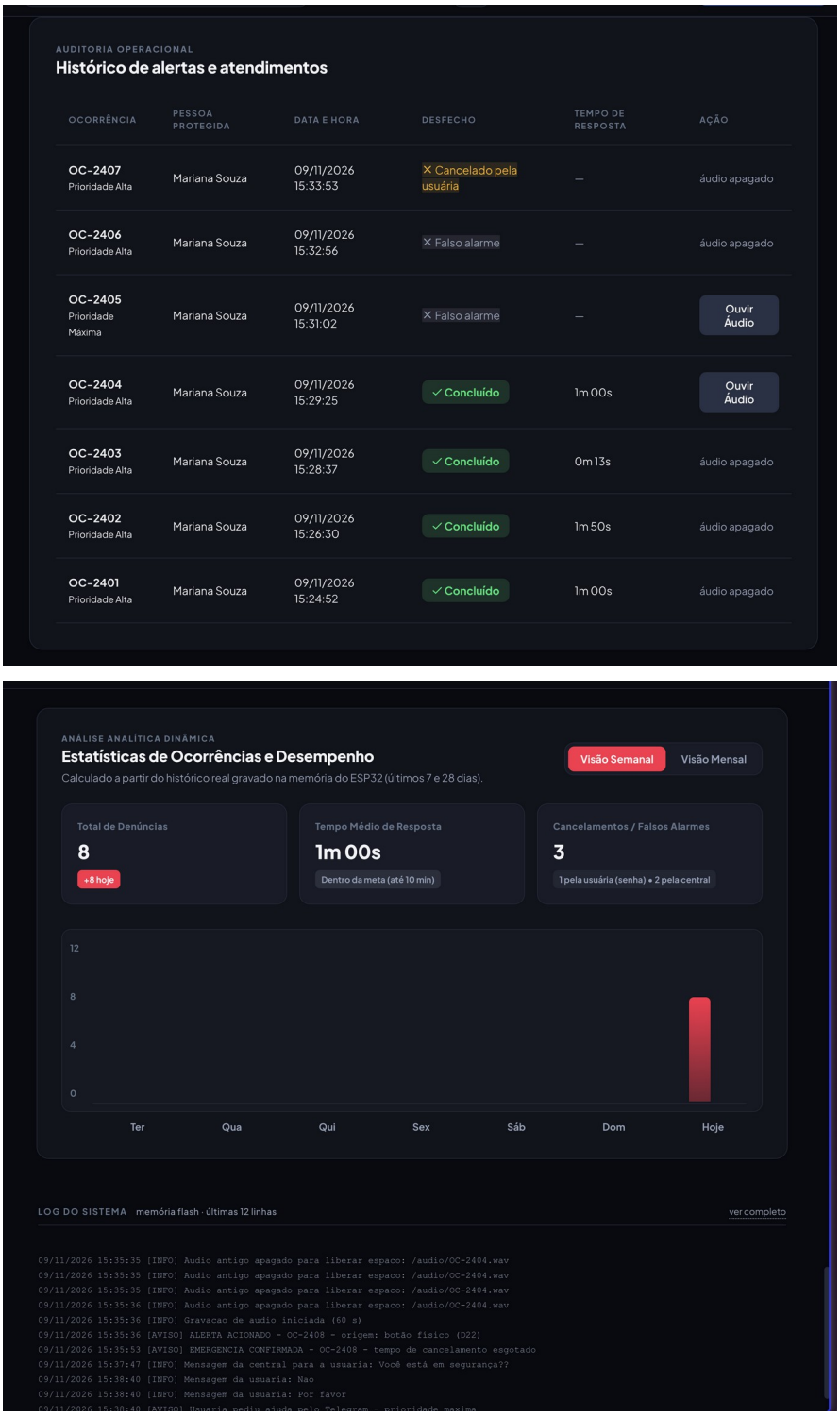
Fonte: Os autores (2026).

Figura 26 – Coação: com a senha de coação, o celular mostra "Alerta cancelado"



Fonte: Os autores (2026).

Figura 27 – Histórico e estatísticas com dados reais

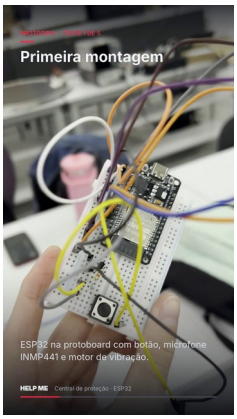


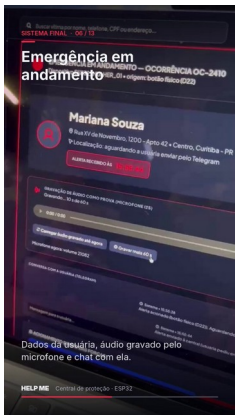
Fonte: Os autores (2026).

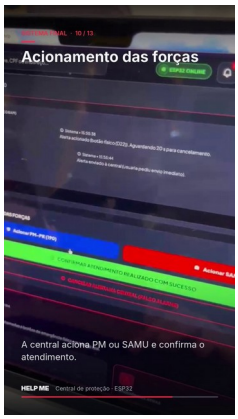
Além dos prints, a equipe gravou um vídeo geral de demonstração, com 2 min 54 s, que mostra o protótipo e o sistema funcionando. Ele está em https://drive.google.com/file/d/1CQPIhX4oUAhRQfv2Jqt-ol8FbV0izJc4/view?usp=drivesdk e no QR code da Figura 28.

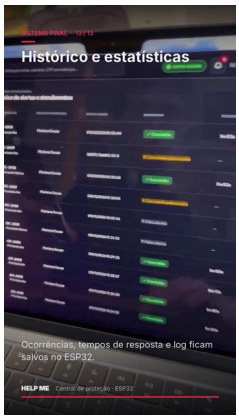
Na primeira parte (0:06 a 0:54), o vídeo mostra a montagem, a leitura do microfone no Monitor Serial, o motor ligando e desligando, o botão acionando o motor e o teste integrado de botão e som alto. Na segunda parte (0:54 a 2:48), mostra o pré-alerta na central, o Telegram da usuária, o cancelamento com senha, um novo acionamento sem cancelamento, a ocorrência com dados da usuária, áudio gravado, chat e posição no mapa, o motor acionado pela central, o acionamento da PM e do SAMU, o encerramento e o histórico com estatísticas.

Figura 28 – Vídeo geral de demonstração: quadros e QR code do link









Fonte: Os autores (2026).

6 RESULTADOS

O Quadro 10 compara cada requisito obrigatório do TDE com o que o HELP ME entrega.

Quadro 10 – Atendimento aos requisitos do TDE

Requisito do TDE	Como o HELP ME atende	Evidência
ESP32 como servidor (listen, aceitar, responder, passivo)	Servidor HTTP na porta 80, sempre em escuta	Seções 2.5 e 3.4; testes de porta e curl
Conectar ao Wi-Fi e reconectar com log de data e hora	Reconexão a cada 10 s, com AVISO e INFO no log	Queda e reconexão do Wi-Fi
Clientes consultam dados do sensor	Rotas /sensor e /status e card ao vivo na central	Rotas no navegador e curl
Clientes enviam comandos	/atuador, /simular, /chat/enviar, /resolver e outras	Motor pela rede
Log persistente na flash com ERRO, AVISO e INFO	LittleFS, com data e hora por NTP, mantido após reinício	Log com alertas e log após reiniciar
IP, MAC, status da conexão e dados em tempo real	Rota /status e central atualizada a cada 1 s	Central e acesso pelo celular
Uso da pilha TCP/IP e de portas	TCP 80 e 443, UDP 123, IPv4 e Wi-Fi	Seção 2.2 e testes
1 sensor e 1 atuador (não LED nem buzzer)	Botão e microfone; motor de vibração	Protótipo montado
Código no GitHub	Repositório da equipe	Seção 4.3

Fonte: Os autores (2026).

A versão final também inclui cancelamento com senha, senha de coação, gravação de áudio como prova, conversa e localização em tempo real pelo Telegram, e histórico com estatísticas calculadas das ocorrências reais. Na primeira rodada, o servidor respondeu a 100 requisições seguidas sem erro e manteve o log depois de reiniciar. Na segunda rodada, em 09/11/2026, o ping ao ESP32 teve 4 respostas em 4 envios, sem perda e com tempo médio de 38,6 ms. A porta 80 aceitou a conexão, a rota /status devolveu o JSON com IP, MAC, rede, sinal e hora, e uma rota inexistente respondeu 404. O motor ligou e desligou pela rede, e o log registrou cada comando com data e hora, inclusive o desligamento automático após 10 s. No fim dos testes, o histórico da central tinha 8 ocorrências, com tempo médio de resposta de 1 min 00 s, entre elas 1 cancelamento pela usuária e 2 falsos alarmes encerrados pela central.

O protótipo tem limitações. Ele funciona ligado ao USB, sem bateria. A central não tem login, e qualquer aparelho da rede local pode abri-la. A conexão com o Telegram não verifica o certificado do servidor, o que só serve para protótipo. A flash comporta poucos minutos de áudio, e o alerta só chega ao Telegram se o Wi-Fi tiver acesso à internet. Os dados cadastrais da vítima são fictícios.

7 CONCLUSÃO

O HELP ME mostrou que um ESP32 e componentes de baixo custo bastam para montar um sistema ciberfísico de alerta que funciona em rede real. O ESP32 atua como servidor HTTP na porta 80, responde aos clientes da rede com os dados dos sensores, aceita comandos para o atuador, mantém um log persistente com data e hora e reconecta sozinho quando o Wi-Fi cai. Com isso, o projeto atende aos requisitos do TDE.

O Telegram e as regras de funcionamento deixaram o projeto mais próximo de uma situação real. A vítima aciona o alerta com um gesto simples, pode cancelar um alarme falso e, sob ameaça, pode fingir o cancelamento com a senha de coação. No desenvolvimento, a equipe também aprendeu a depurar problemas de hardware e de sistema embarcado, como a partição da flash e a ordem de inicialização das tarefas.

A case 3D do dispositivo ficou pronta em 10/11/2026. Como próximos passos, a equipe sugere alimentar o dispositivo por bateria, adicionar login à central, verificar o certificado TLS do Telegram e estudar a detecção automática de gritos pelo microfone como forma extra de acionamento.

REFERÊNCIAS

BRASIL. **Lei nº 11.340, de 7 de agosto de 2006**. Cria mecanismos para coibir a violência doméstica e familiar contra a mulher. Brasília, DF: Presidência da República, 2006. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2006/lei/l11340.htm. Acesso em: 7 nov. 2026.

CHOUDHARY, Nikita. **Smart Safety Device**. [S. l.]: ElectronicWings, [20--]. Disponível em: https://www.electronicwings.com/users/NikitaChoudhary/projects/6466/smart-safety-device. Acesso em: 7 nov. 2026.

EDDY, Wesley (ed.). **RFC 9293**: Transmission Control Protocol (TCP). [S. l.]: Internet Engineering Task Force, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9293. Acesso em: 7 nov. 2026.

ESPRESSIF SYSTEMS. **Arduino-ESP32 documentation**. [S. l.], [20--]. Disponível em: https://docs.espressif.com/projects/arduino-esp32/en/latest/. Acesso em: 7 nov. 2026.

FIELDING, Roy T.; NOTTINGHAM, Mark; RESCHKE, Julian (ed.). **RFC 9110**: HTTP Semantics. [S. l.]: Internet Engineering Task Force, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110. Acesso em: 7 nov. 2026.

FÓRUM BRASILEIRO DE SEGURANÇA PÚBLICA. **Anuário Brasileiro de Segurança Pública 2026**. São Paulo: FBSP, 2026. Disponível em: https://forumseguranca.org.br/anuario-brasileiro-seguranca-publica/. Acesso em: 7 nov. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 7498-1**: Information technology – Open Systems Interconnection – Basic Reference Model: the basic model. Geneva: ISO, 1994.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a Internet**: uma abordagem top-down. 8. ed. São Paulo: Pearson, 2021.

LEE, Edward A.; SESHIA, Sanjit A. **Introduction to embedded systems**: a cyber-physical systems approach. 2. ed. Cambridge: MIT Press, 2017.

POSTEL, Jon. **RFC 768**: User Datagram Protocol. [S. l.]: Internet Engineering Task Force, 1980. Disponível em: https://www.rfc-editor.org/rfc/rfc768. Acesso em: 7 nov. 2026.

SANTOS, Rui. **ESP32 Web Server – Arduino IDE**. [S. l.]: Random Nerd Tutorials, [20--]a. Disponível em: https://randomnerdtutorials.com/esp32-web-server-arduino-ide/. Acesso em: 7 nov. 2026.

SANTOS, Rui. **Telegram**: ESP32 motion detection with notifications (Arduino IDE). [S. l.]: Random Nerd Tutorials, [20--]b. Disponível em: https://randomnerdtutorials.com/telegram-esp32-motion-detection-arduino/. Acesso em: 7 nov. 2026.

SANTOS, Sara. **ESP32 Datalogger**: download data file via web server (Arduino IDE). [S. l.]: Random Nerd Tutorials, 2024. Disponível em: https://randomnerdtutorials.com/esp32-datalogger-download-data-file/. Acesso em: 7 nov. 2026.

SANTOS, Sara. **ESP32 with INMP441 I2S omnidirectional microphone module (Arduino IDE)**. [S. l.]: Random Nerd Tutorials, 2026. Disponível em: https://randomnerdtutorials.com/esp32-inmp441-i2s-microphone-arduino/. Acesso em: 7 nov. 2026.

TANENBAUM, Andrew S.; WETHERALL, David. **Redes de computadores**. 5. ed. São Paulo: Pearson Prentice Hall, 2011.

TELEGRAM. **Telegram Bot API**. [S. l.], [20--]. Disponível em: https://core.telegram.org/bots/api. Acesso em: 7 nov. 2026.

VASOYA, Kamlesh. **Smart SOS device with emergency alert button and alarm**. [S. l.]: Electronics For You, 2022. Disponível em: https://www.electronicsforu.com/electronics-projects/smart-sos-device-raises-alarm-sends-location. Acesso em: 7 nov. 2026.

APÊNDICE A – PINAGEM DO ESP32

HELP ME — Pinagem do ESP32

ESP32 DevKit V1 (30 pinos) · protoboard de 30 linhas · atualizado em 04/11/2026 · soldagem: Letícia Beatriz Silva de Morais

1. Peças do projeto (nomenclatura técnica)

Peça	Nome técnico / modelo	Função no projeto
Microcontrolador	**ESP32 DevKit V1** (30 pinos) · conversor USB-serial **CP2102**	Processamento, Wi-Fi, servidor HTTP; alimentado pelo USB
Microfone	**INMP441** (microfone digital MEMS, interface I2S)	Captação de áudio do alerta
Transistor	**2N2222A** (NPN, encapsulamento TO-92)	Aciona o motor a partir do sinal do D4
Resistor de base	Resistor de ~3,3 kΩ (valor ideal calculado: 1 kΩ)	Limita a corrente do D4 na base do transistor
Botão	Chave táctil (*push button*) de 4 pernas	Acionamento de emergência no D22
Motor de vibração	Mini motor vibratório tipo moeda (*coin motor*) de 3 V	Aviso discreto ao acionar o alerta
Protoboard	Protoboard de 30 linhas (colunas a–j, trilhos + e −)	Base de montagem do circuito
Jumpers	Macho-macho (protoboard) e macho-fêmea (microfone)	Interligação entre os componentes
Cabo USB	Cabo USB para o ESP32	Alimentação 5 V e gravação do código

2. Pinos do ESP32 em uso

Como ler: segure a placa com o **USB para cima** e leia as etiquetas pela frente. O ESP32 está encaixado com os pinos da **esquerda na coluna b** e os da **direita na coluna j**; o 1º pino (junto ao USB) fica na **linha 1**. Como a coluna b está ocupada pelo pino, o furo livre para o jumper é o da **coluna a**, na mesma linha (a1 = mesma ligação do 3V3). A coluna j inteira é ocupada pelos pinos da direita, então **nenhum pino do lado direito é usado**.

Pino	Função no projeto	Posição na placa (lado esq., contando do USB)	Furo da protoboard	Liga em
3V3	Alimentação 3,3 V	1º	b1 (pino) · **a1** (jumper)	Trilho **+**
GND	Terra	2º	b2 (pino) · **a2** (jumper)	Trilho **−**
D4	Sinal do motor de vibração	5º	b5 (pino) · **a5** (jumper)	Fio amarelo → **a26** (resistor → base do 2N2222A)
D18	Microfone INMP441 — SCK	9º	b9 (pino) · **a9** (jumper)	Pino SCK do microfone
D19	Microfone INMP441 — WS	10º	b10 (pino) · **a10** (jumper)	Pino WS do microfone
D21	Microfone INMP441 — SD	11º	b11 (pino) · **a11** (jumper)	Pino SD do microfone
D22	Botão de emergência (INPUT_PULLUP)	14º	b14 (pino) · **a14** (jumper)	Fio → **a28** (perna do botão)

3. Ordem dos pinos do lado esquerdo (do USB para baixo)

Linha	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Pino	**3V3**	**GND**	D15	D2	**D4**	RX2	TX2	D5	**D18**	**D19**	**D21**	RX0	TX0	**D22**	D23

Fundo preto: pinos em uso. Nada deve ser ligado em **a3, a4, a6, a7, a8, a12, a13 e a15**.

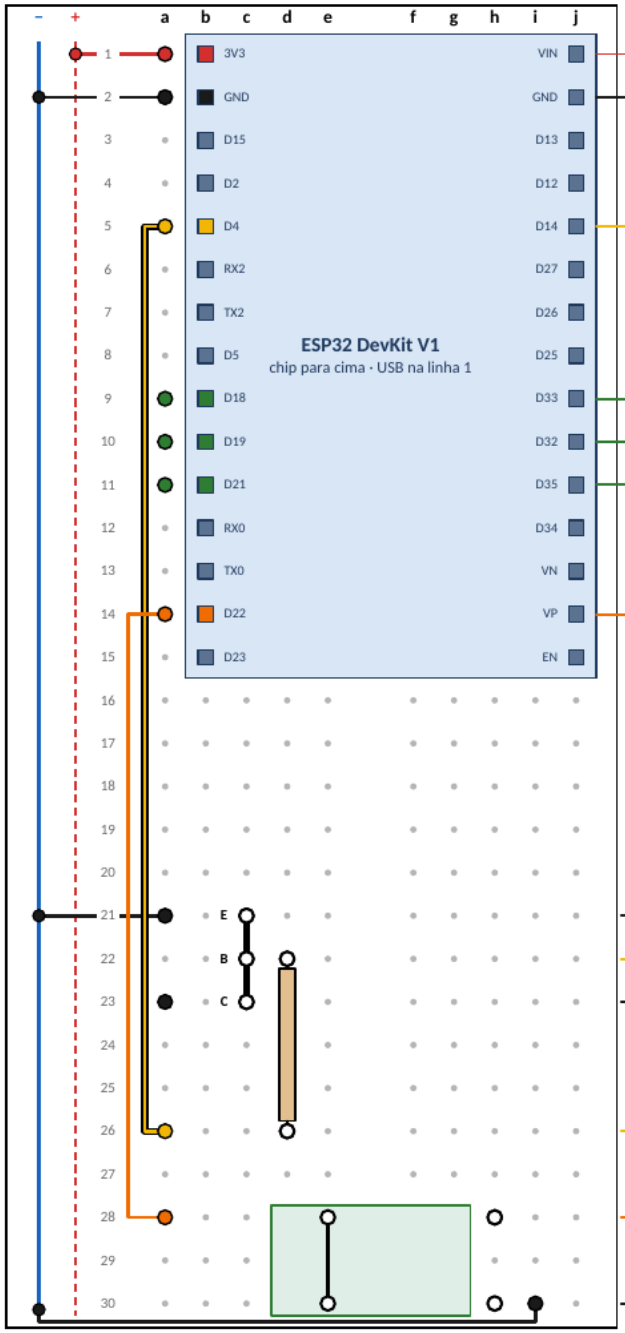
4. Alimentação e regras de ouro

- O **VDD do microfone vai só no 3V3** (trilho +). Nunca no VIN/5 V — queima o microfone.
- Motor de vibração: positivo no trilho **+**; negativo no **coletor** do transistor (a23). O D4 só comanda a base.
- O D26 foi o pino planejado no início, mas **o definitivo é o D4** (por causa do espaçamento da protoboard).
- Pinos a evitar para novas ligações: D0, D2, D12, D15 (afetam a inicialização) e D34, D35, VP, VN (só entrada).
- Se a luz do ESP32 ficar fraca ao ligar o USB: desconecte e suspeite de curto (ponte de solda no microfone, VDD/GND trocados).

Projeto	Documento	Elaboração	Soldagem	Data	Folha
HELP ME	**Pinagem ESP32 e protoboard**	**João Pedro Pires Antunes**	**Letícia Beatriz Silva de Morais**	**04/11/2026**	**1 de 3**

HELP ME — Mapa da protoboard

Vista de cima · linha 1 junto ao USB · colunas a–e (esq.) | vão | f–j (dir.)



3V3 · a1 → trilho +

GND · a2 → trilho –

D4 · a5 → a26 (fio amarelo)

Símbolos
- pino do ESP32
- furo com jumper ou fio
- furo com componente

Cores
- 3,3 V
- GND
- sinal D4
- microfone
- botão D22

D18 · a9 ← SCK do microfone

D19 · a10 ← WS do microfone

D21 · a11 ← SD do microfone

D22 · a14 → a28 (botão)

Trilho +
jumper do a1 · fio VERMELHO do motor · VDD do microfone

Trilho –
jumper do a2 · fio preto (a21) · GND do botão · GND e L/R do microfone

Cada linha de a–e (e de f–j) é um único ponto elétrico. Os furos f–i das linhas 1–15 estão sob a placa do ESP32.

Transistor E (c21) · a21 ← trilho – (fio preto)

Transistor B (c22) · resistor começa em d22

Transistor C (c23) · a23 ← fio preto do motor

Microfone INMP441: solto, ligado por 6 jumpers macho-fêmea.

Resistor termina em d26 · a26 ← fio amarelo (D4)

Botão (perna) · a28 ← fio do D22

Botão (perna) · i30 → trilho – (GND)

Projeto	Documento	Elaboração	Soldagem	Data	Folha
HELP ME	**Pinagem ESP32 e protoboard**	**João Pedro Pires Antunes**	**Letícia Beatriz Silva de Morais**	**04/11/2026**	**2 de 3**

HELP ME — Lista de fios e componentes

Use esta página para conferir cada cabo se algo der errado

1. Jumpers e fios na protoboard

#	Fio	Ponta 1	Ponta 2	Para quê
1	Jumper 3V3	a1	Trilho +	Leva 3,3 V ao trilho
2	Jumper GND	a2	Trilho –	Leva o terra ao trilho
3	Jumper preto	Trilho –	a21	Emissor do transistor ao GND
4	Jumper amarelo	a5 (D4)	a26	Sinal D4 → resistor → base
5	Motor — fio vermelho	Trilho +	—	Positivo do motor (3V3)
6	Motor — fio preto	a23	—	Negativo do motor ao coletor
7	Jumper do botão (D22)	a14	a28	Sinal do botão no D22
8	Jumper do botão (GND)	i30	Trilho –	GND do botão

2. Componentes encaixados na protoboard

Peça	Onde fica	Detalhe
Transistor 2N2222A	Coluna **c**, linhas **21 / 22 / 23**	Com as letras viradas para você: E = linha 21 · B = linha 22 · C = linha 23
Resistor de base	Coluna **d**: **d22 → d26**	Liga a base (linha 22) ao fio do D4 (linha 26). Valor usado: ~3,3 kΩ (estoque); o ideal calculado era 1 kΩ
Botão de emergência	Atravessa o vão, **linhas 28 e 30**	Uma perna na linha 28 (D22) e outra na linha 30 (GND); apertar liga as duas
Motor de vibração	Fora da placa, 2 fios soldados em jumpers	Vermelho → trilho + · preto → a23

3. Microfone INMP441 (fora da protoboard, 6 jumpers macho-fêmea)

Pino do microfone	Ponta macho vai em	Pino do ESP32 / alimentação
SCK	a9	D18
WS	a10	D19
SD	a11	D21
VDD	Trilho +	3V3 — **nunca** em VIN/5 V
GND	Trilho –	GND
L/R	Trilho –	GND (canal esquerdo)

A ponta **fêmea** de cada jumper encaixa no pino do microfone; a ponta **macho** vai na protoboard.

4. Conferência rápida antes de ligar o USB

- ☐ Trilho + tem só: jumper do a1, fio vermelho do motor e VDD do microfone.
- ☐ Trilho – tem: jumper do a2, fio preto (a21), GND do botão (i30), GND e L/R do microfone.
- ☐ Nenhum fio do trilho – encosta em algum fio do trilho +.
- ☐ Fio amarelo: a5 ↔ a26. Fio do botão: a14 ↔ a28. Fio preto do motor: a23.
- ☐ Pernas do transistor cada uma sozinha em sua linha (21, 22, 23).
- ☐ Nada ligado em a3, a4, a6, a7, a8, a12, a13 e a15.

Pinos usados no código (helpme_final): motor = D4 · botão = D22 (INPUT_PULLUP) · microfone SCK = D18, WS = D19, SD = D21.

Projeto	Documento	Elaboração	Soldagem	Data	Folha
HELP ME	**Pinagem ESP32 e protoboard**	**João Pedro Pires Antunes**	**Letícia Beatriz Silva de Morais**	**04/11/2026**	**3 de 3**